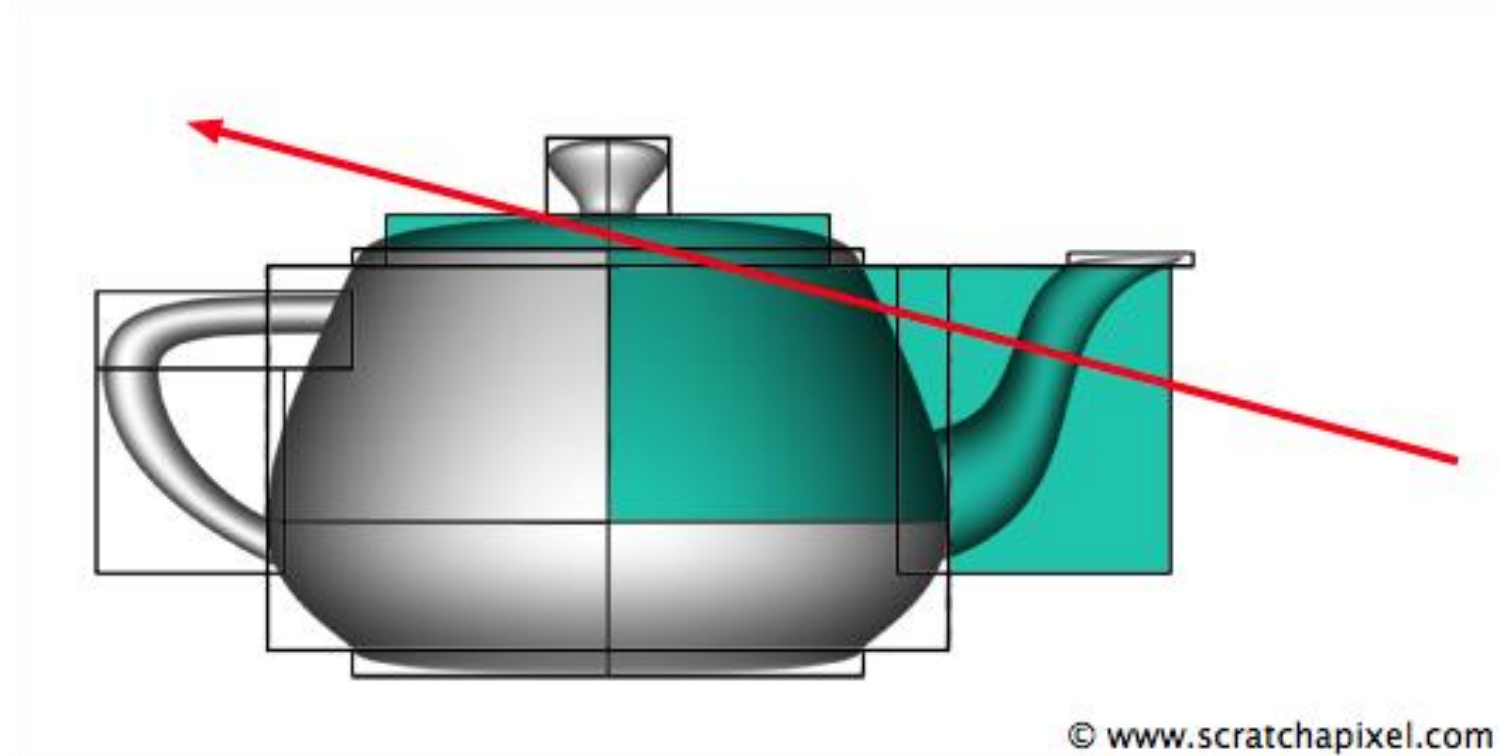


Bounding Volume Hierarchies



Some Slides/Images adapted from Karan Singh, Marschner and Shirley and David Levin

Agenda

- Motivation: Common Geometric Queries in Graphics
- Bounding Volumes
 - Spheres
 - Boxes (AABB, OOBB)
- Constructing Object-Partitioning Hierarchies
 - Sphere Trees
 - AABB Trees
- Space-Partitioning Hierarchies
 - Uniform Spatial Subdivision
 - Axis-Aligned Spatial Subdivision

Closest Point

Closest point on a triangle?

Project p to p' in plane of triangle.

Calculate barycentric coordinates of p' .

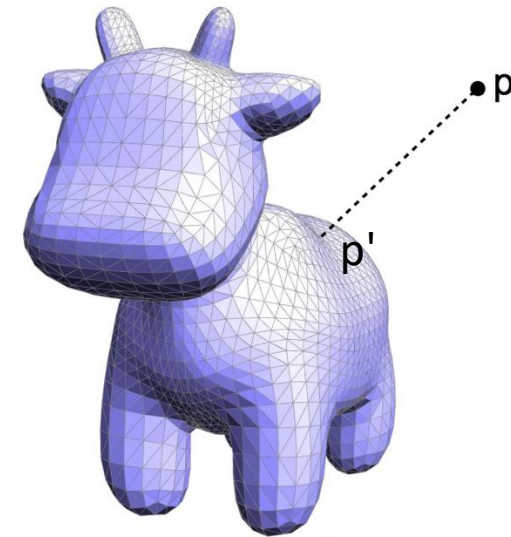
Clamp coordinates to $[0,1]$.

Point with clamped coordinates is closest to p .

Loop over all triangles and keep the closest.

What is the complexity?

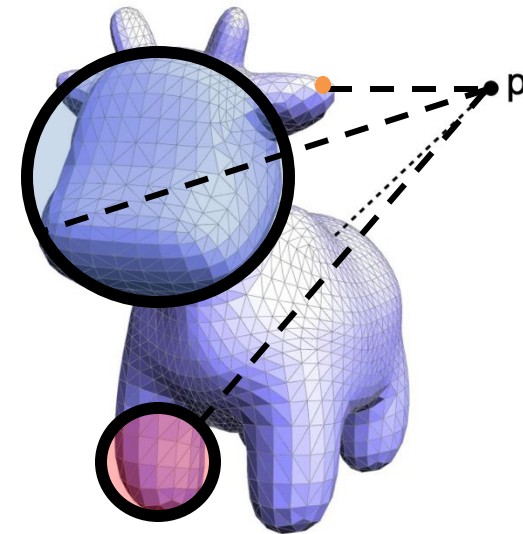
What if the object has a billion triangles?



What point on the mesh is closest to p ?

Geometric modeling and geometric queries

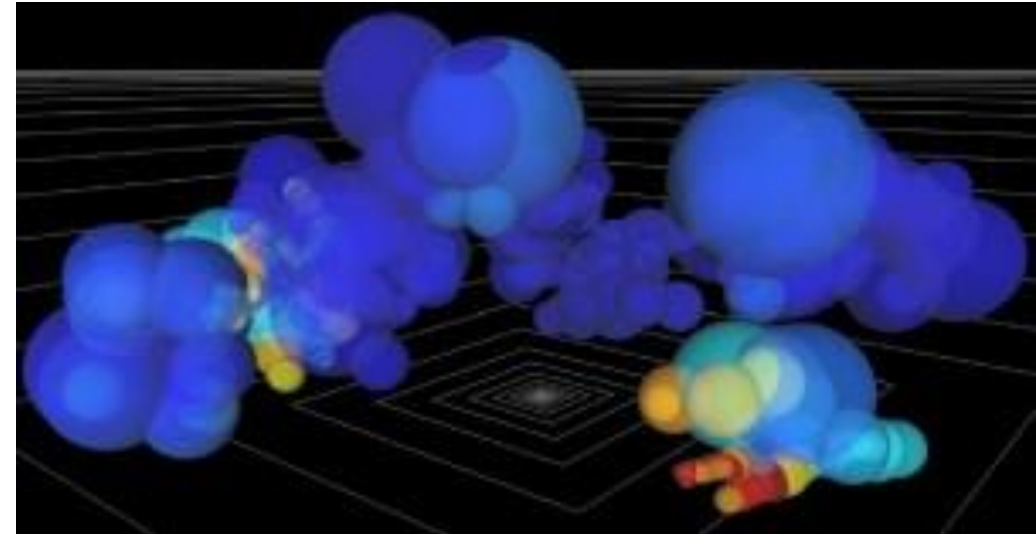
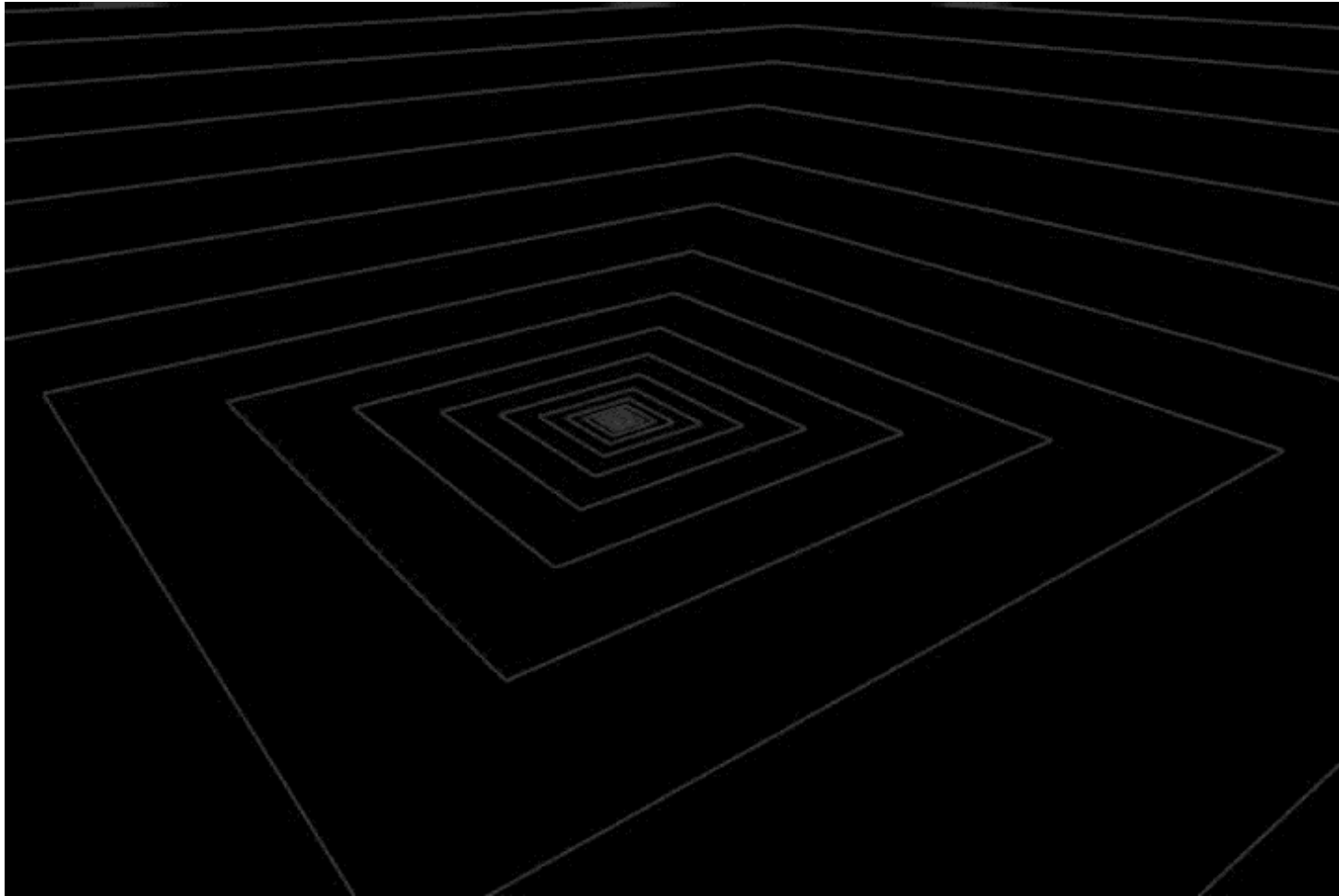
- Closest/furthest point to a sphere with center c and radius r is:
 $c \pm r * \text{normalized } (p-c)$
- Closest point on red sphere is further than furthest point on blue sphere.
- Closest point on blue sphere is further than the orange point on a triangle.

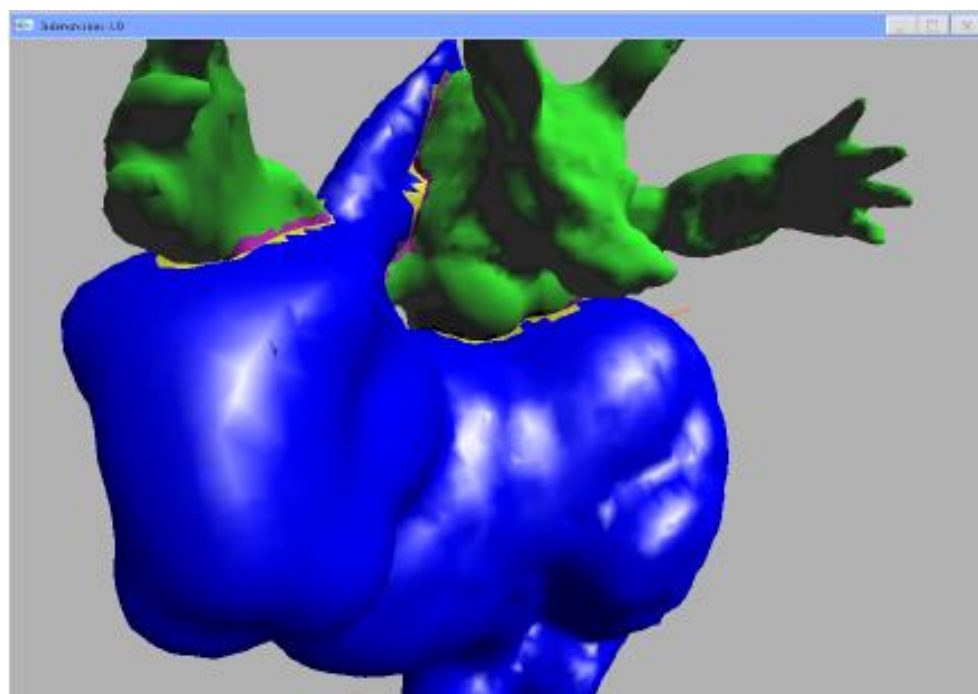
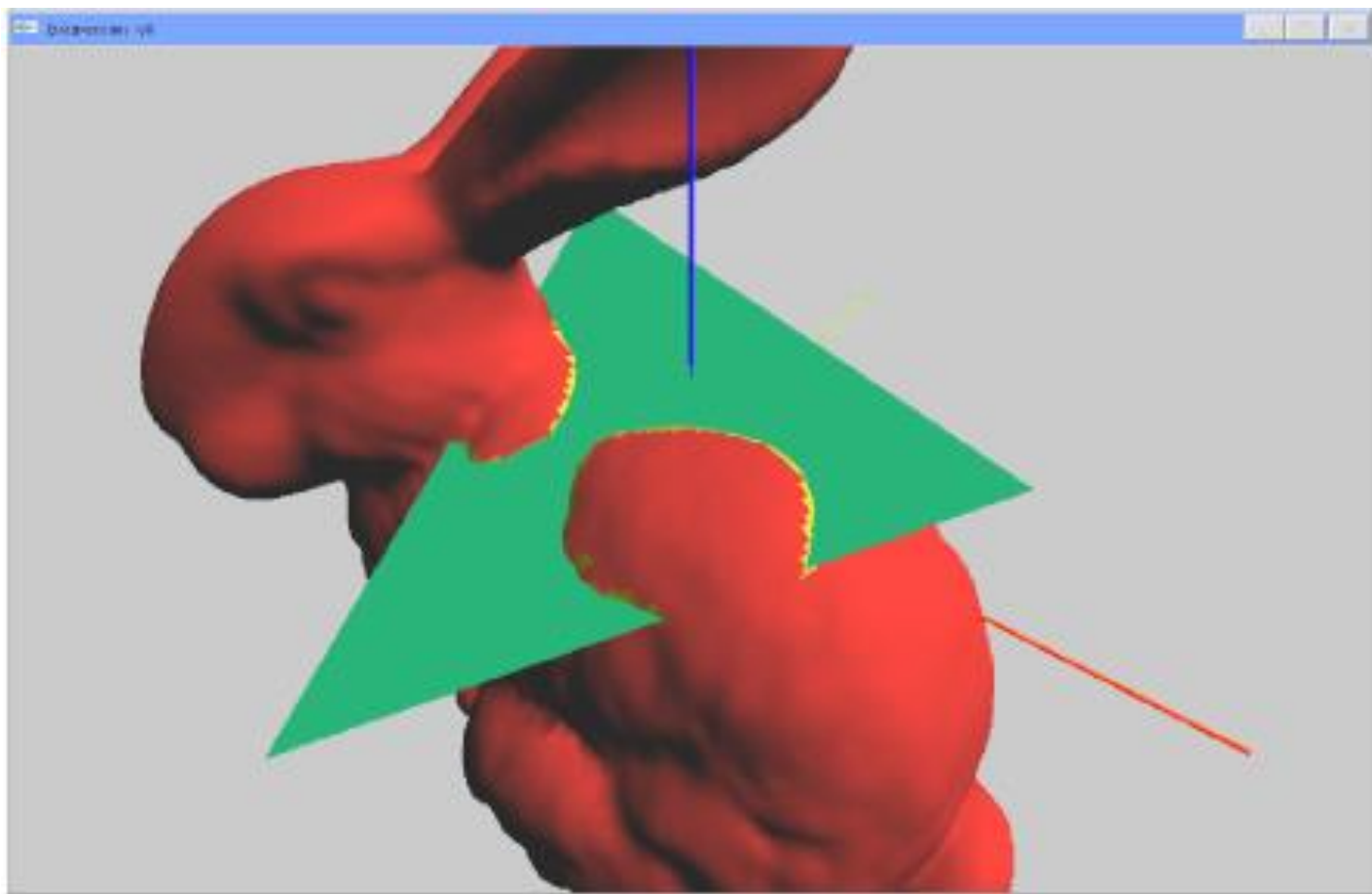


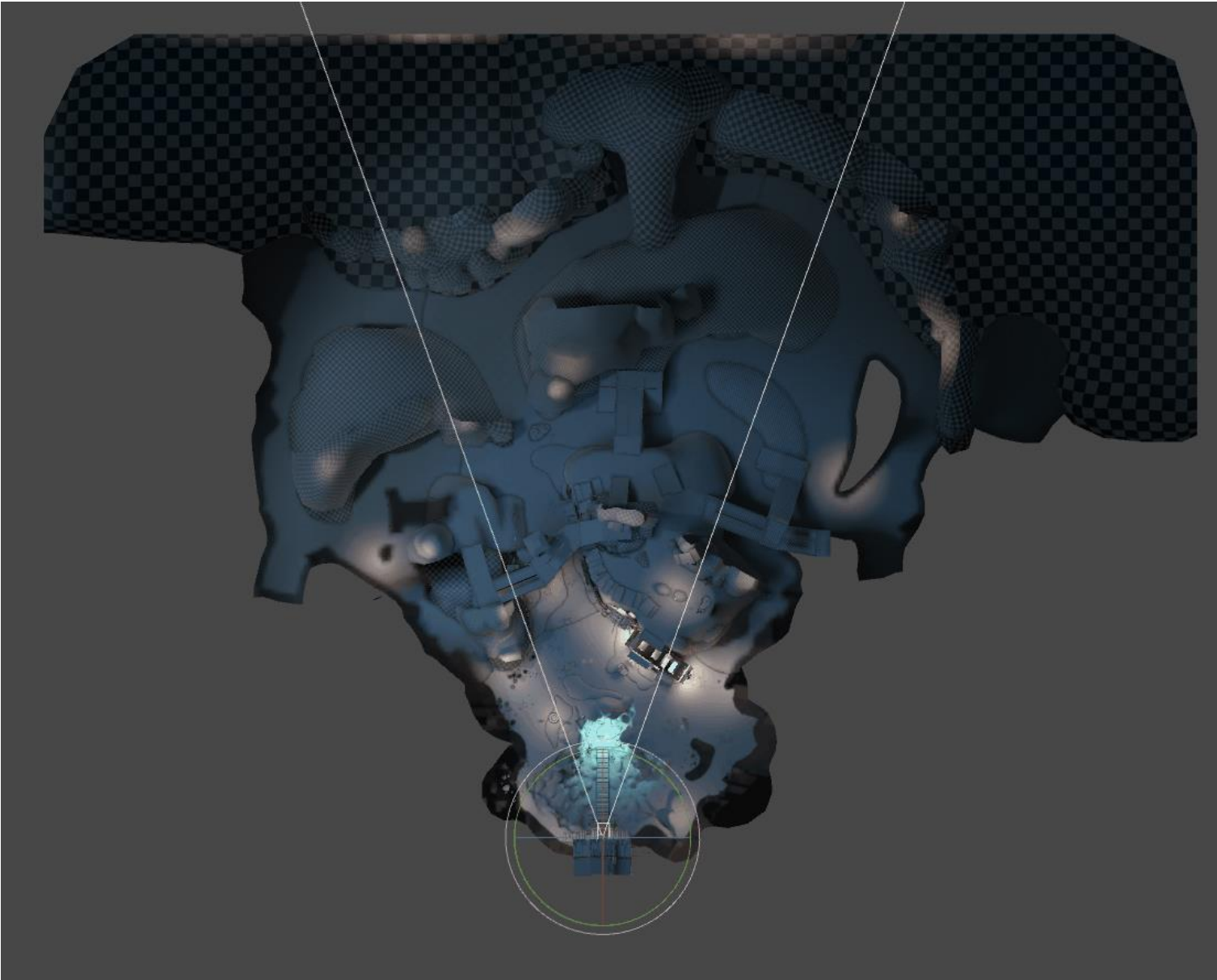
What point on the mesh is closest to p ?

Collision Intersection

How can you test if two spheres intersect?





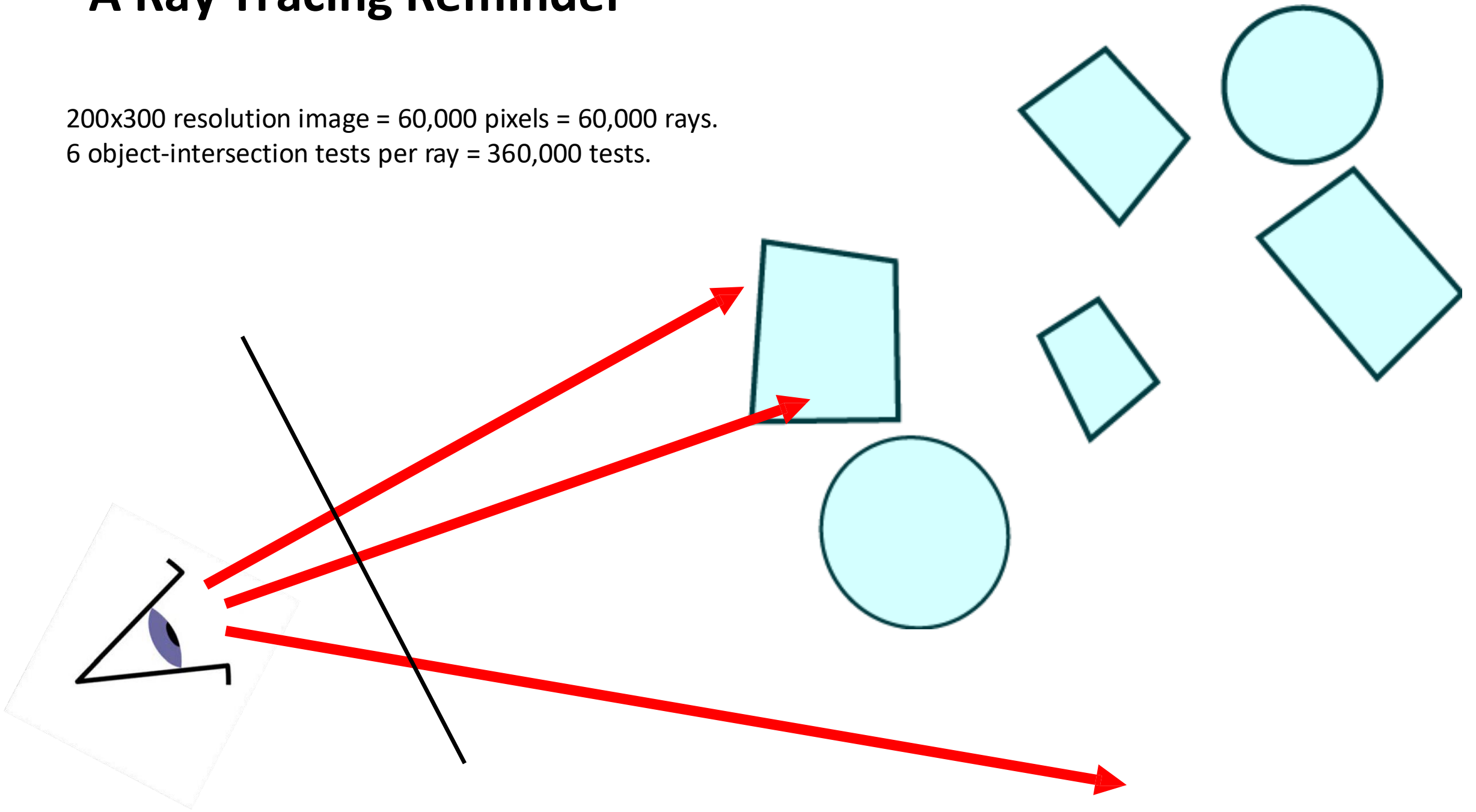


Frustum Culling: Loading only the models in the game you see.

[ArtStation - Home a Drone - Technical Breakdown](#)

A Ray Tracing Reminder

200x300 resolution image = 60,000 pixels = 60,000 rays.
6 object-intersection tests per ray = 360,000 tests.



Ray Intersection With BVs

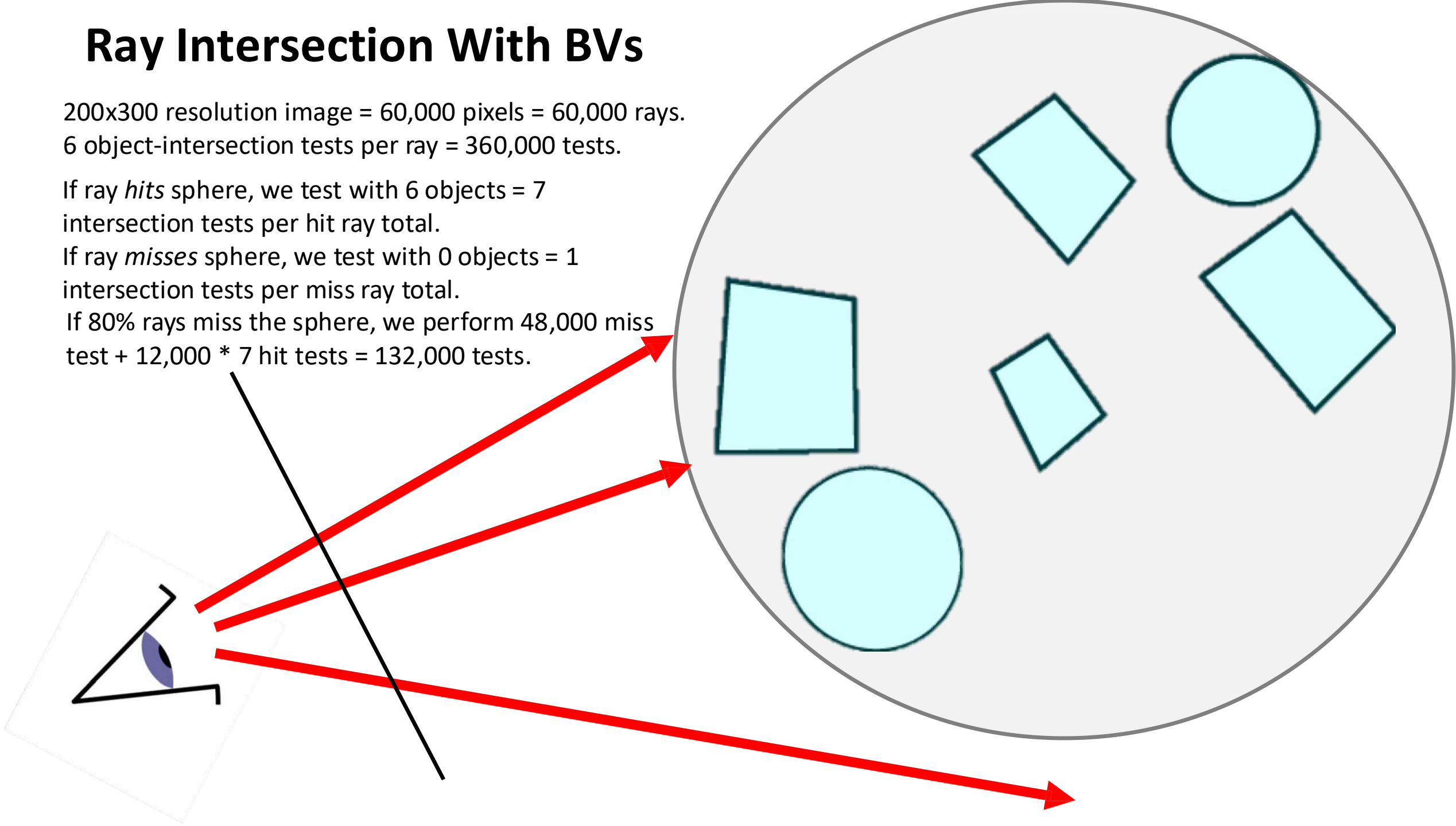
200x300 resolution image = 60,000 pixels = 60,000 rays.

6 object-intersection tests per ray = 360,000 tests.

If ray *hits* sphere, we test with 6 objects = 7 intersection tests per hit ray total.

If ray *misses* sphere, we test with 0 objects = 1 intersection tests per miss ray total.

If 80% rays miss the sphere, we perform 48,000 miss test + 12,000 * 7 hit tests = 132,000 tests.



Bounding volumes

Quick way to avoid expensive testing intersections and collisions:

 bound object with a simple volume **bvol** (volume encloses object)

If a ray/object doesn't hit/intersect **bvol**,

 it doesn't hit/intersect the object

else

 test object for hit/intersect

Cost: more for hits and near misses, less for far misses

Worth doing? Yes if:

1. Cost of **bvol** intersection test is small: simple shapes (spheres, boxes, ...)
2. Cost of object intersect test is large: **bvol** most useful for complex objects
3. Tightness of fit of **bvol** to object is good:

 Loose fit leads to extra object intersections

 Tradeoff between tightness and **bvol** intersection cost

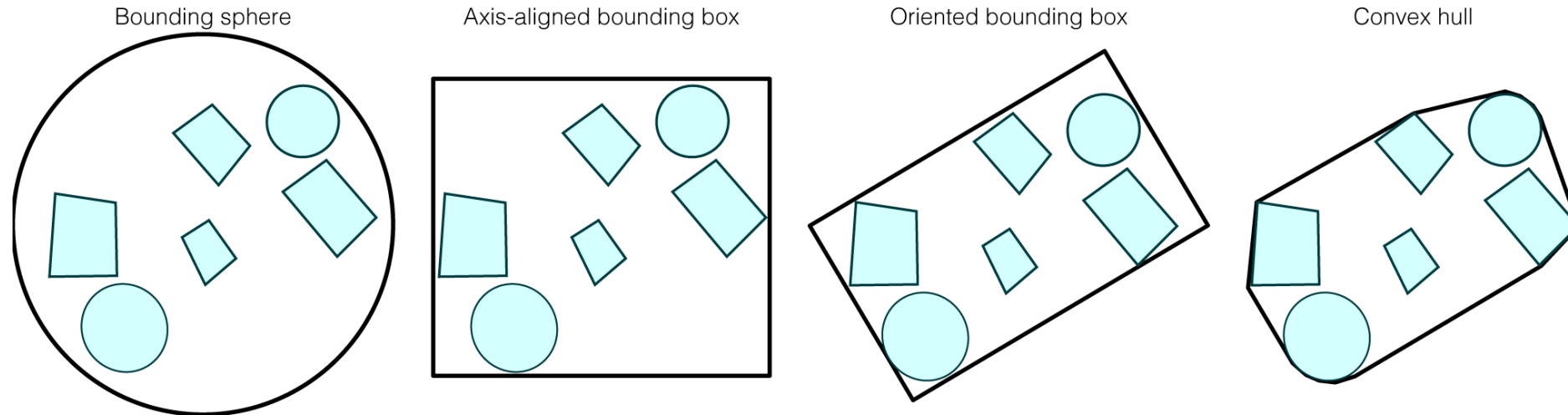
Choice of bounding volumes

Spheres: easy to intersect, not always tight.

Axis-aligned Bounding Boxes (AABBs): easy to intersect, tighter for axis-aligned objects.

Oriented bounding boxes (OBBs): easy to intersect (transformation cost),
tighter than AABBs.

Convex Hull: not as easy to intersect as the above, tighter than the above.

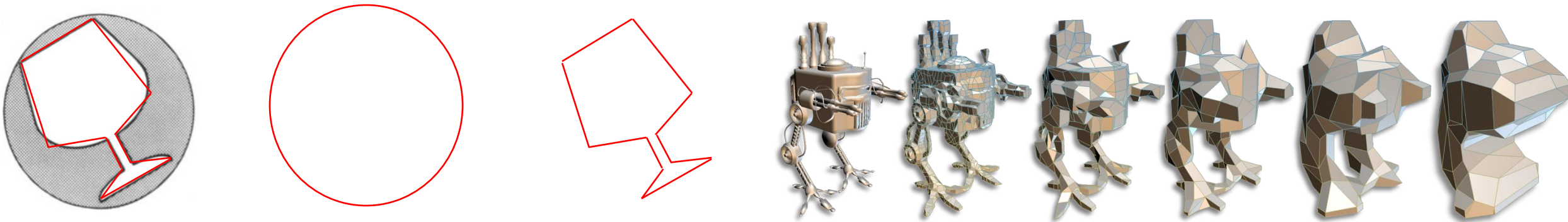


Proxy Geometry vs. Bounding volumes

Another concept often used is proxy geometry or Level-Of-Detail LOD.

A proxy is a simplified representation of the object, that can be used as the object when rendering and processing speed is more important than visual accuracy.

Note, that proxy geometry is an approximation of the object and typically not a bounding volume. And a bounding volume itself is typically not a good visual proxy for an object.



Close and Far

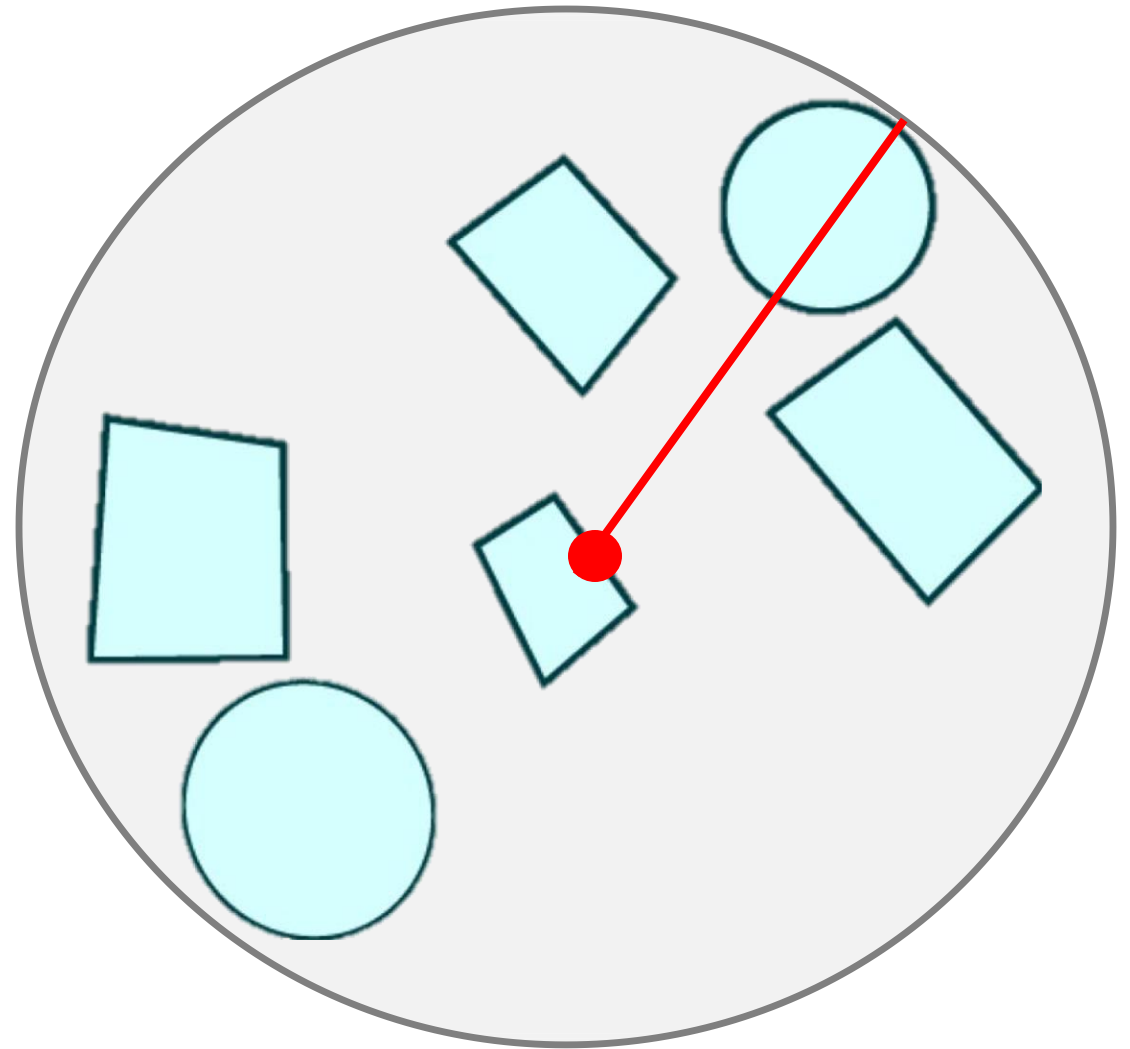


Building a Bounding Sphere

Parameters of a Sphere:

1. Center = $\mathbf{c} = \frac{1}{n} \sum_{i=1}^n \mathbf{v}^i$
2. Radius = $r = \max(\|\mathbf{v}^i - \mathbf{c}\|)$

$\mathbf{v}^i \in \text{Vertices}$



Ray-Sphere Intersection

Substitute ray equation into implicit equation for sphere

$$(\mathbf{e} + t\vec{\mathbf{d}} - \mathbf{c}) \cdot (\mathbf{e} + t\vec{\mathbf{d}} - \mathbf{c}) - r^2 = 0$$

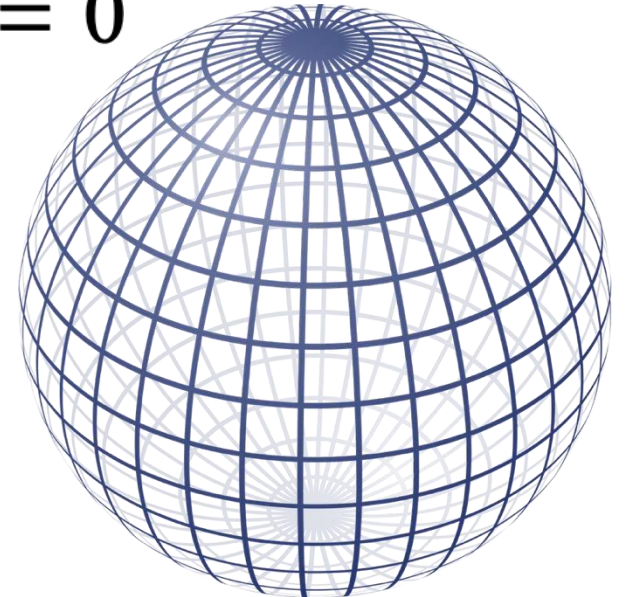
Rearrange

$$(\vec{\mathbf{d}} \cdot \vec{\mathbf{d}})t^2 + 2\vec{\mathbf{d}} \cdot (\mathbf{e} - \mathbf{c})t + (\mathbf{e} - \mathbf{c}) \cdot (\mathbf{e} - \mathbf{c}) - r^2 = 0$$

Looks familiar...

$$At^2 + Bt + C = 0$$

It's a quadratic! (can use the quadratic equation)



Axis Aligned Bounding Boxes (AABB)

Surround objects with a box (2D) or cube (3D)

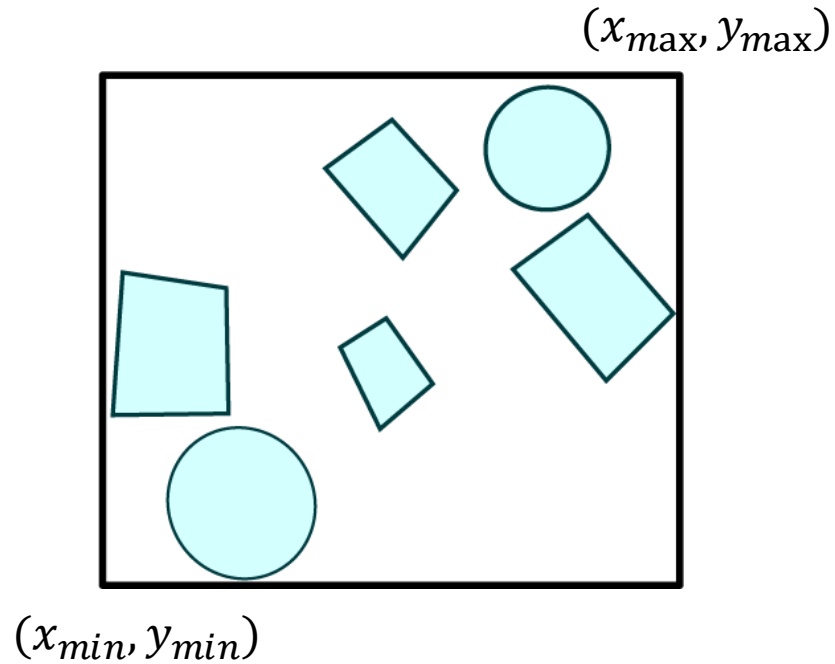
$$x_{\min} = \min(v_x^i)$$

$$x_{\max} = \max(v_x^i)$$

$$y_{\min} = \min(v_y^i)$$

$$y_{\max} = \max(v_y^i)$$

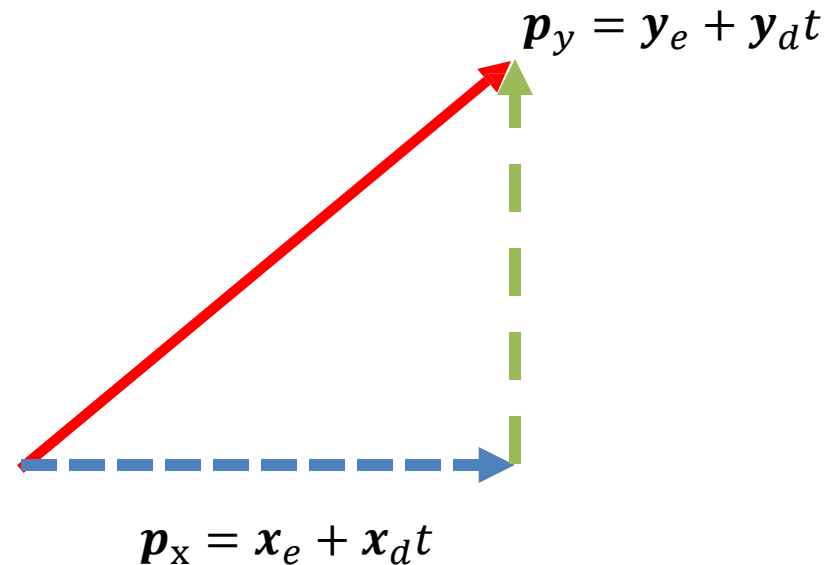
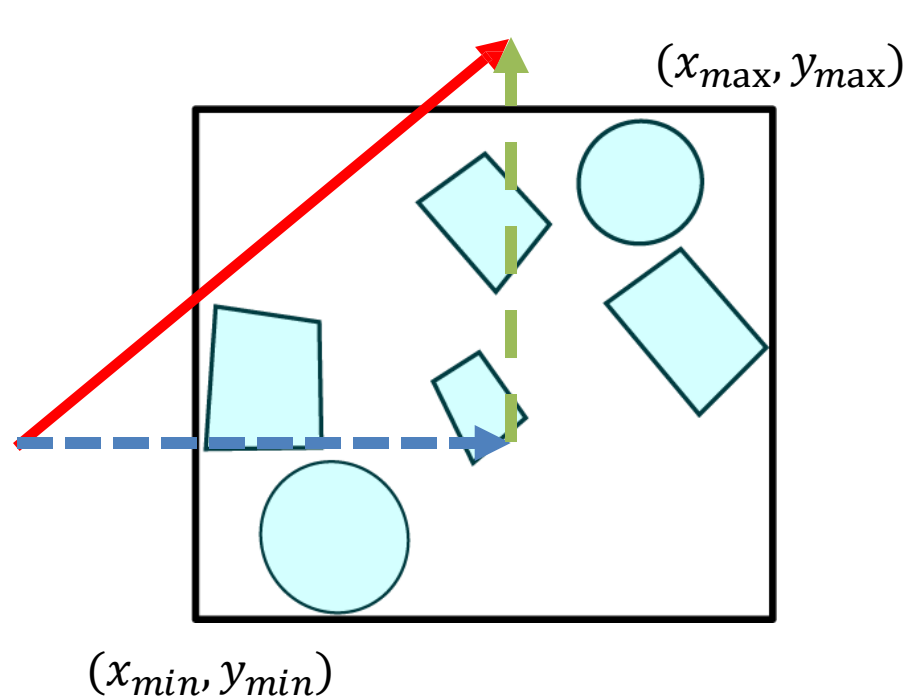
$\mathbf{v}^i \in \text{Vertices}$



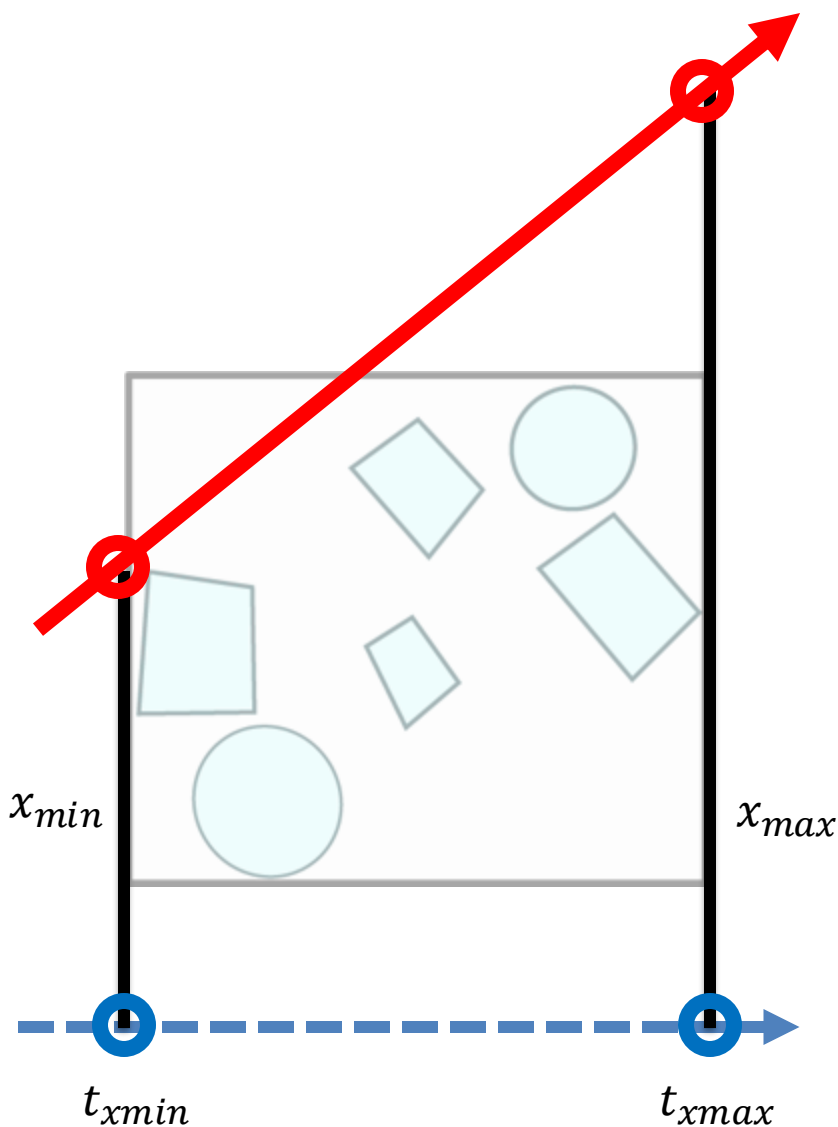
Ray-AABB Intersection

How to intersect an AABB with a ray $p(t) = e + d \cdot t$

Separate problem into x-component intersection and y-component intersection.



Ray-AABB Intersection

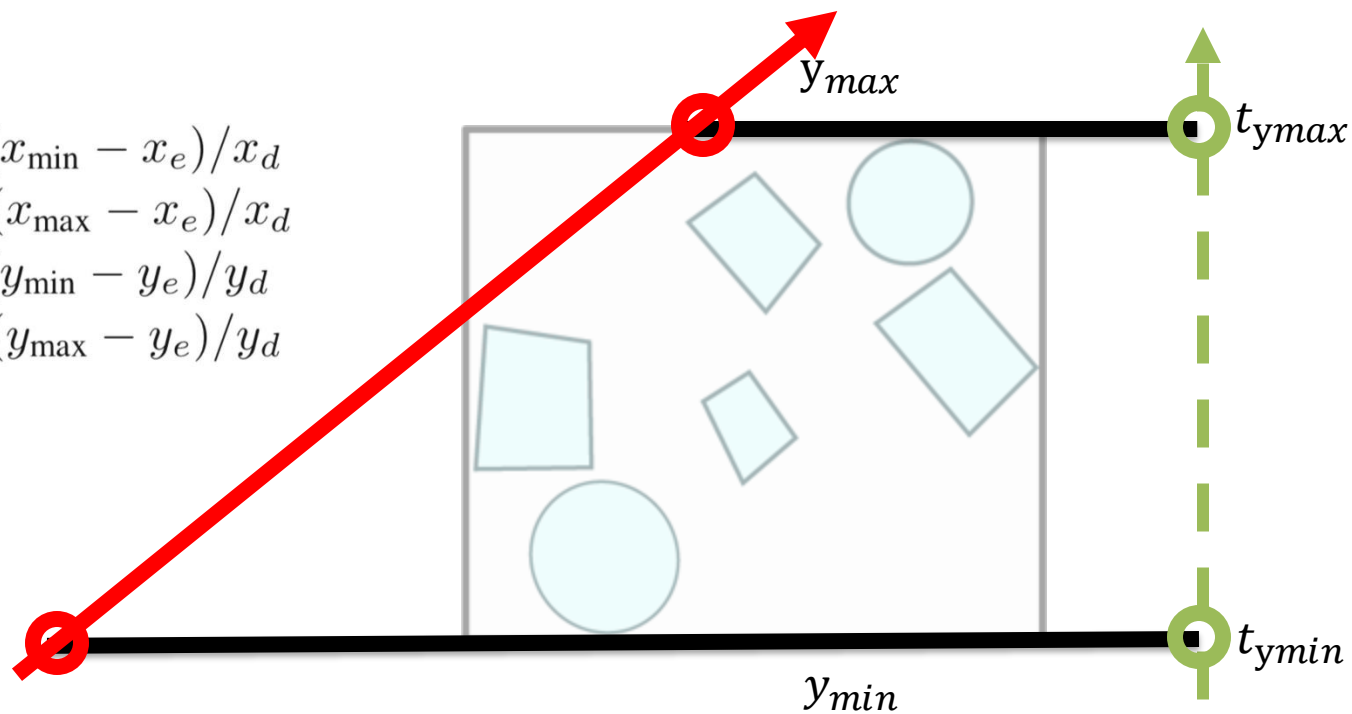


$$t_{xmin} = (x_{min} - x_e) / x_d$$

$$t_{xmax} = (x_{max} - x_e) / x_d$$

$$t_{ymin} = (y_{min} - y_e) / y_d$$

$$t_{ymax} = (y_{max} - y_e) / y_d$$



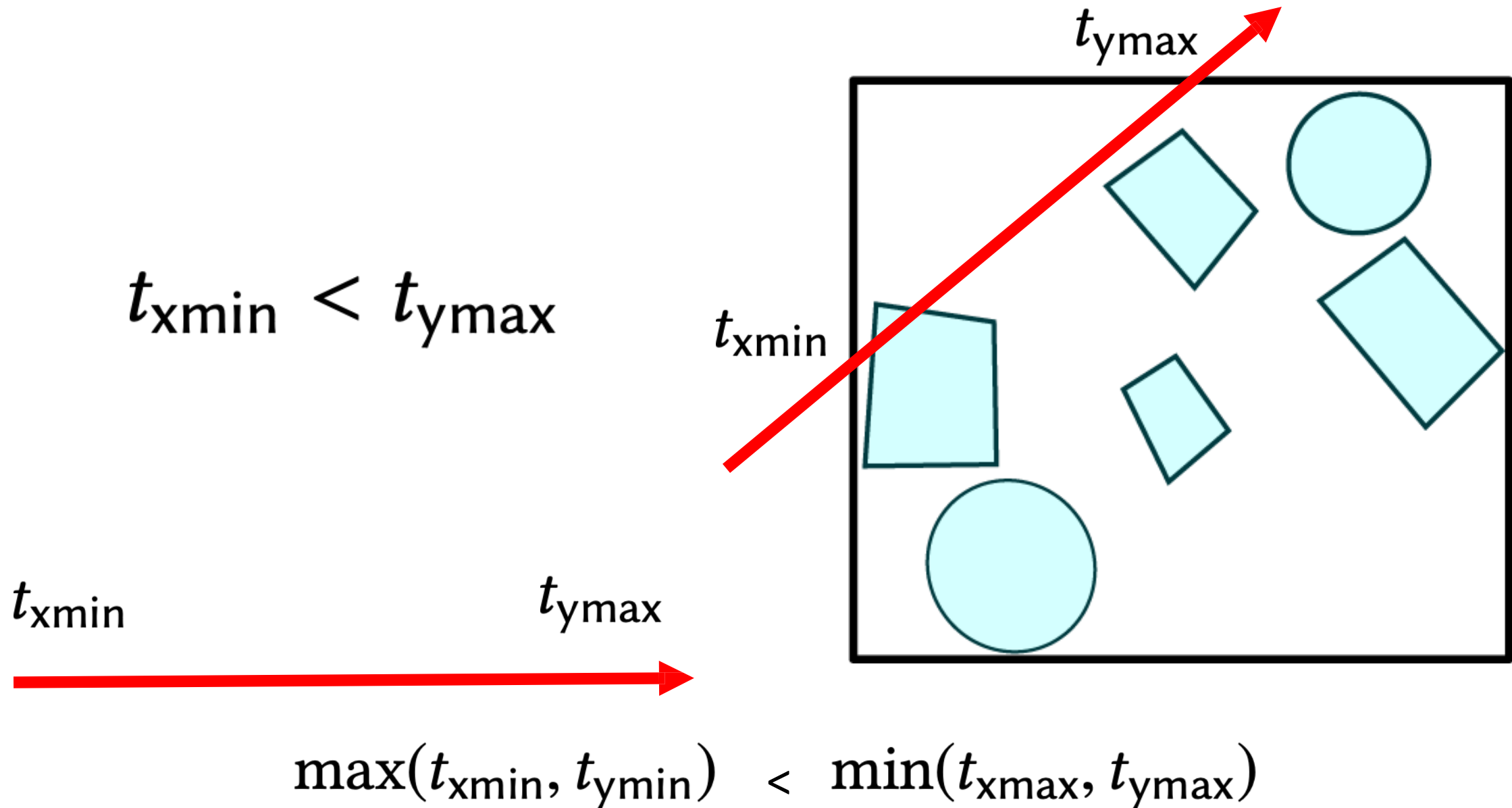
$$t \in [t_{xmin}, t_{xmax}]$$

$$t \in [t_{ymin}, t_{ymax}]$$

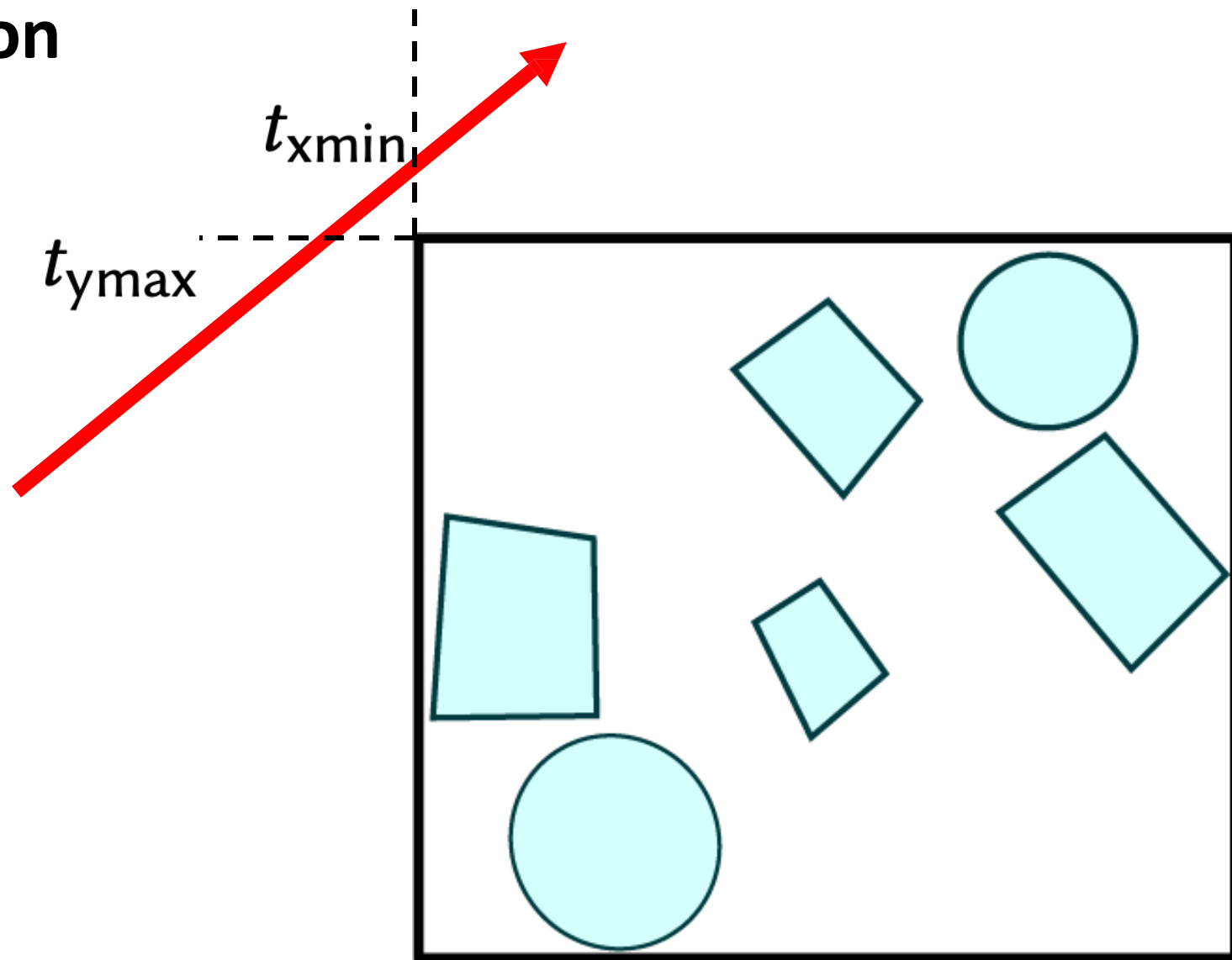
$$t \in [t_{xmin}, t_{xmax}] \cap [t_{ymin}, t_{ymax}]$$

$$t_{xmin} < t_{ymax}$$

Ray-AABB Intersection

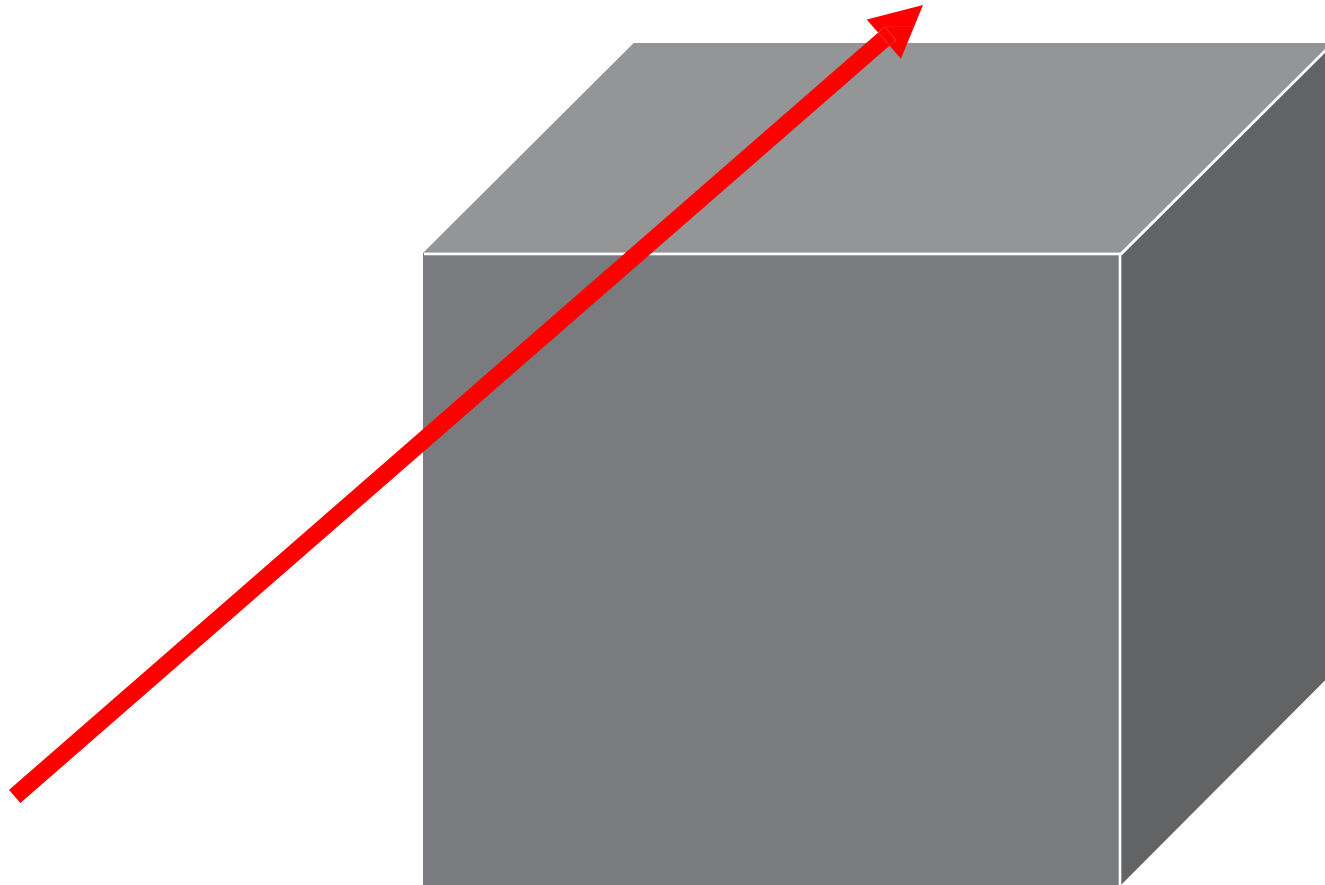


Ray-AABB Intersection

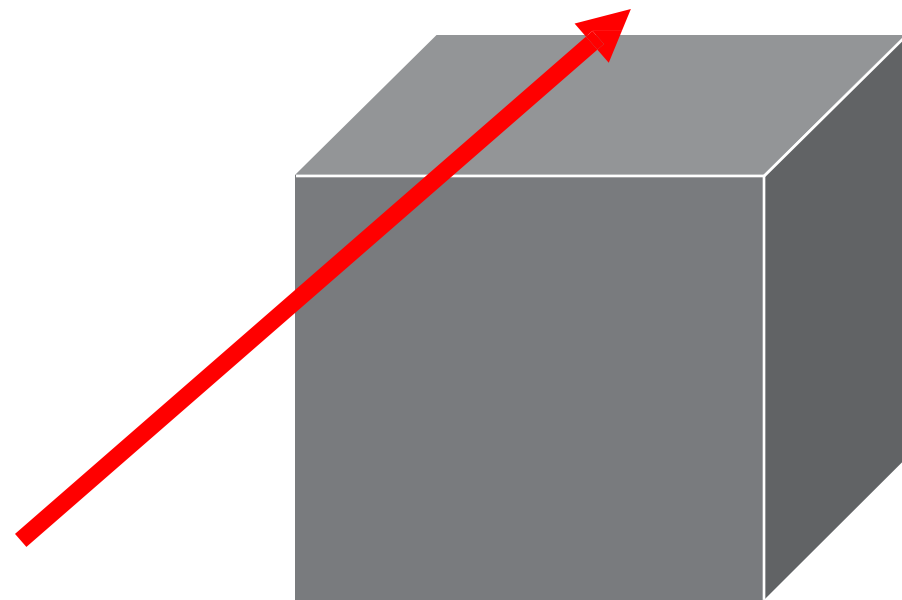
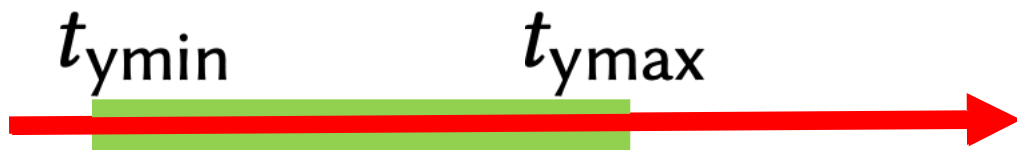
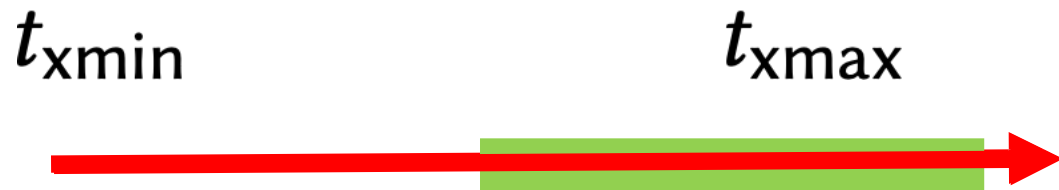


$$\max(t_{xmin}, t_{ymin}) < \min(t_{xmax}, t_{ymax})$$

What happens in 3D ?

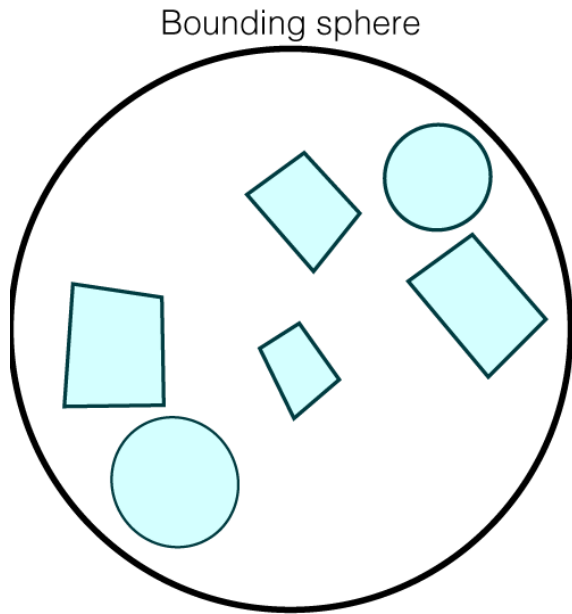


Ray-AABB Intersection

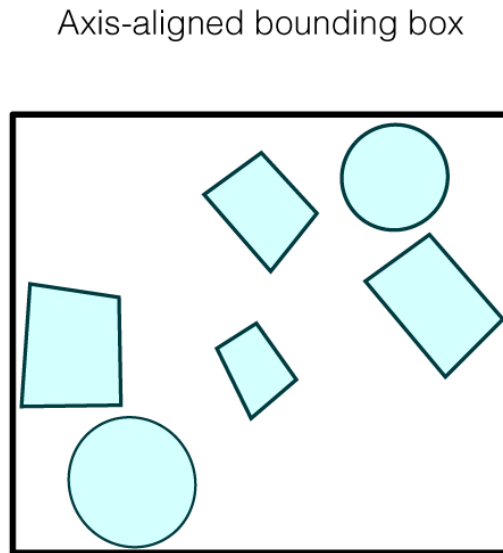


Bounding Volumes (BVs)

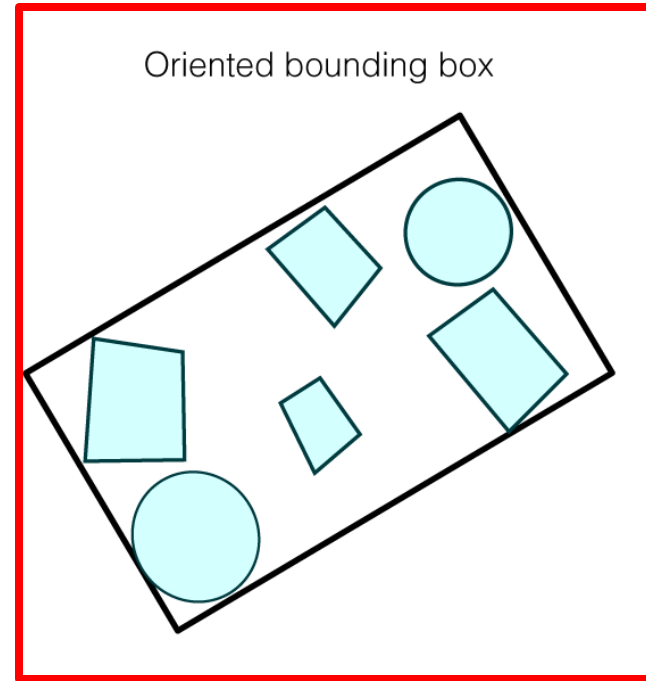
“Simple” geometry that fully encloses a **collection** of other geometry



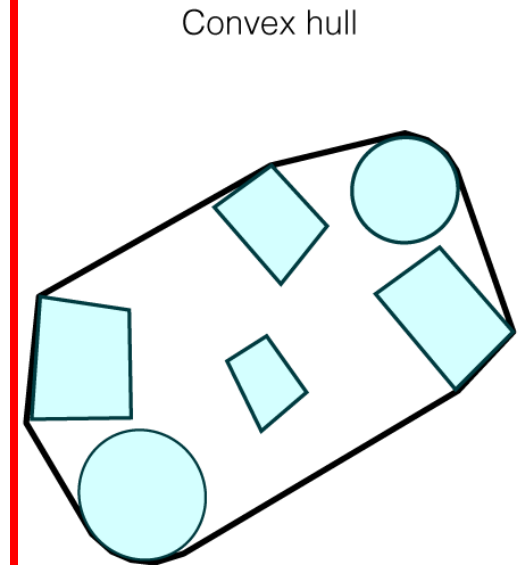
Sphere
(a lot)



AABB
(a lot)



OBB
(a little)



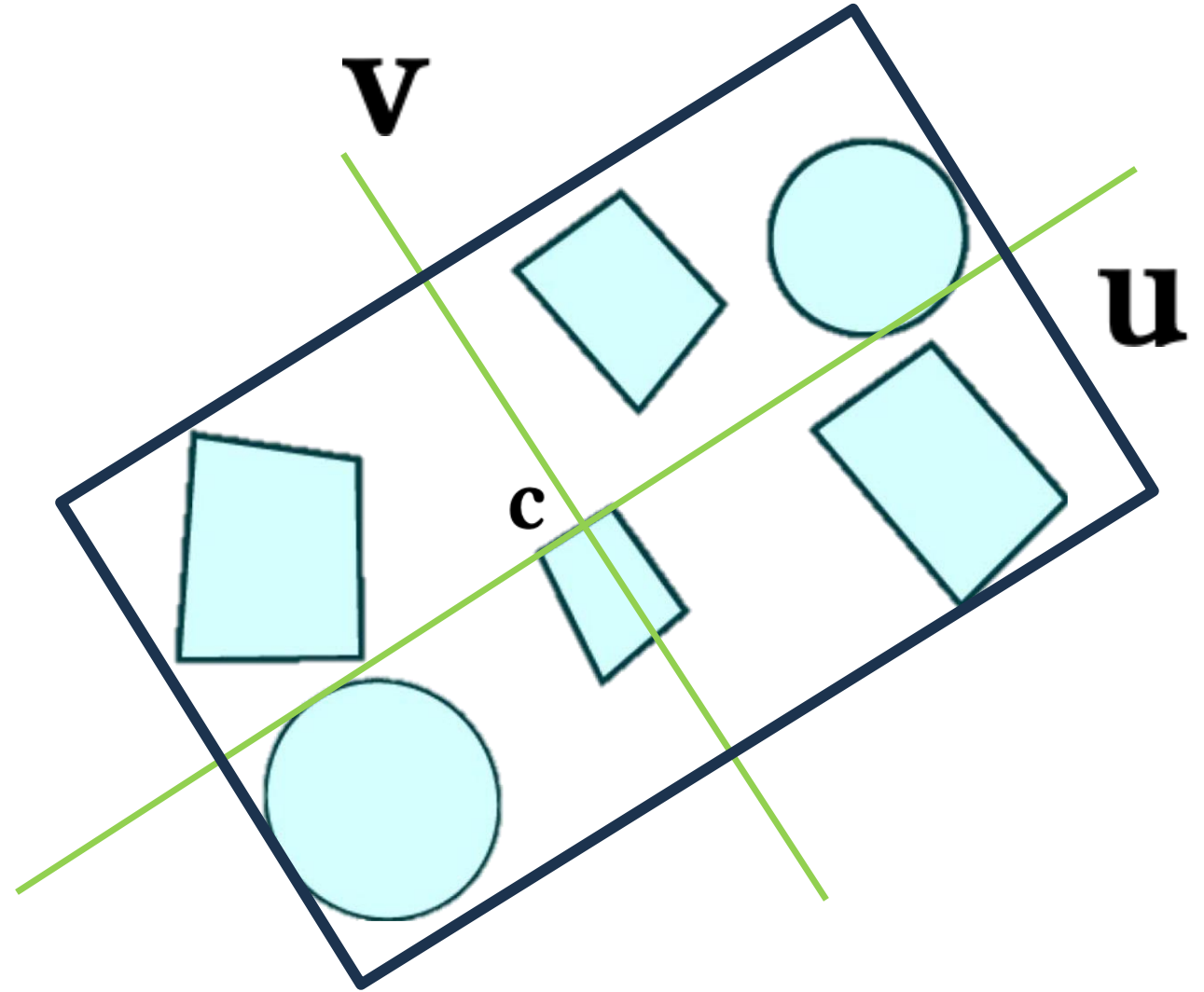
Convex Hull
(nope!)

Building an Object-Oriented Bounding Box (OOBB)

$$\mathbf{c} = \frac{1}{n} \sum_{i=1}^n \mathbf{v}^i$$

Principle Axes

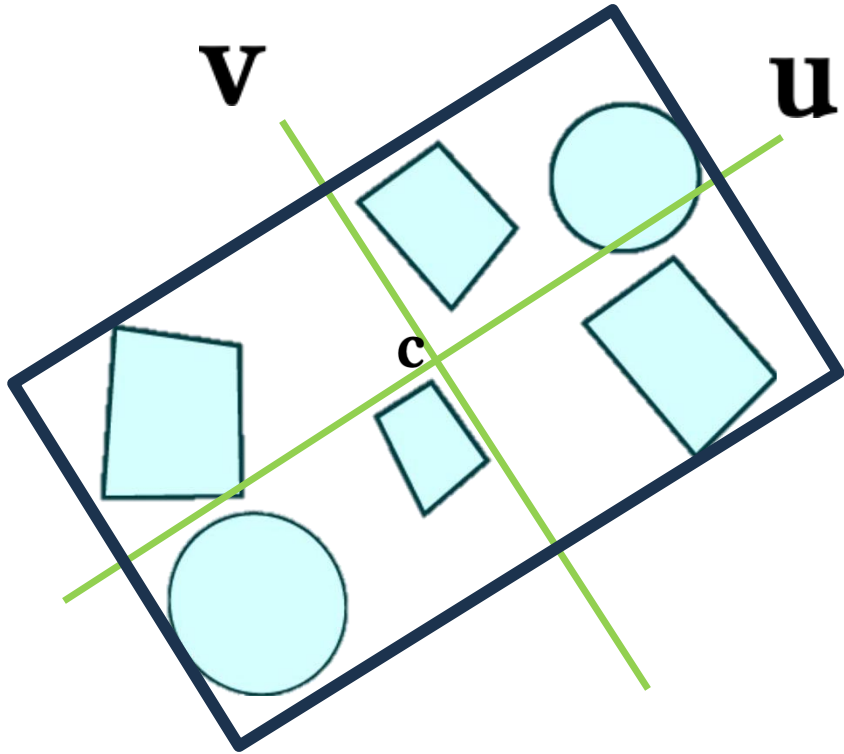
$\begin{bmatrix} \mathbf{u} & \mathbf{v} \end{bmatrix}$



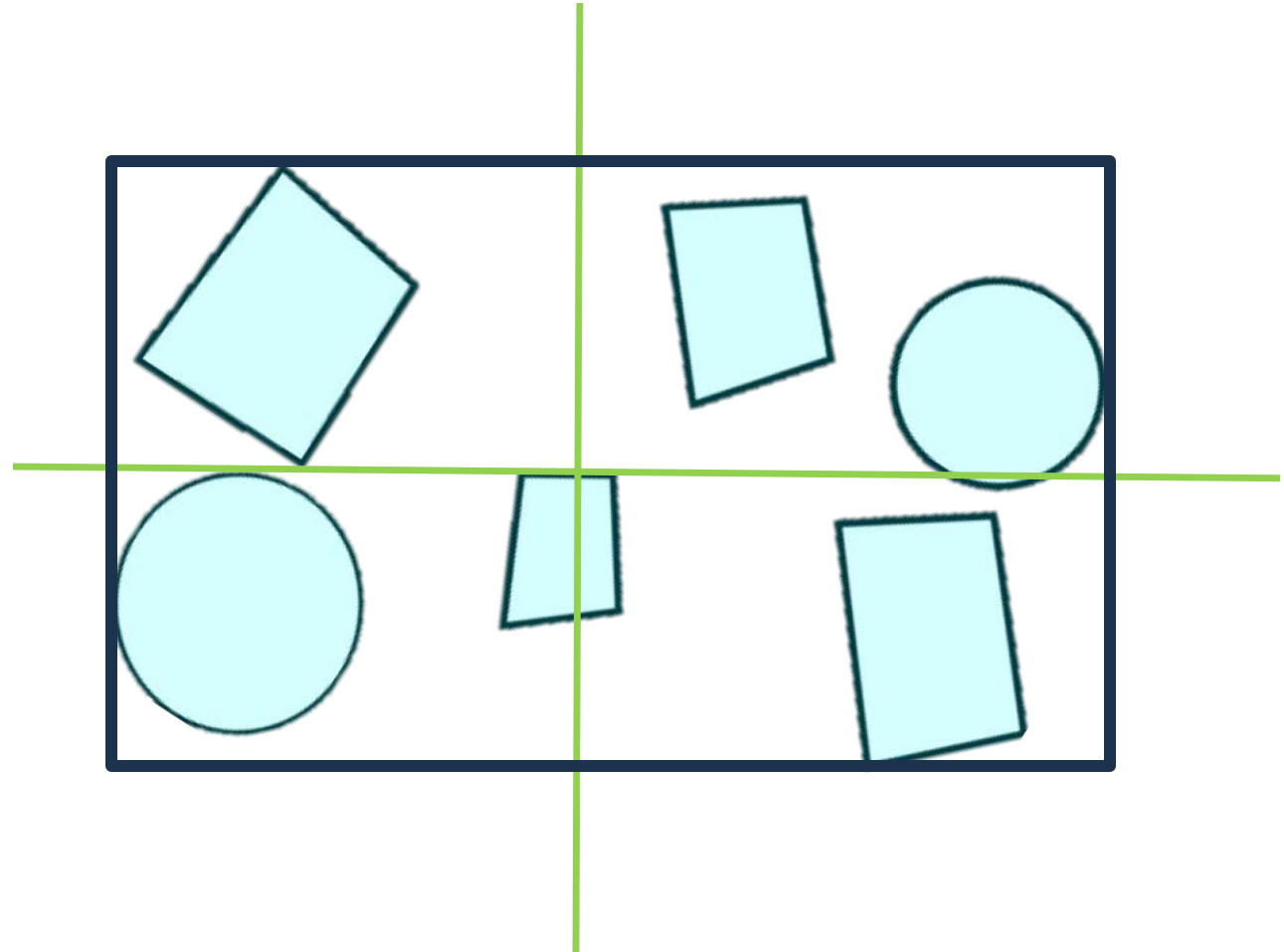
Find directions of maximum and minimum variance

With PCA

Building an Object-Oriented Bounding Box (OOBB)



Build Rotation Matrix $\mathbf{R}$



Implementing a bounding volume

Add new Surface subclass, ***BoundedSurface***

Contains a ***bvol*** and a reference to a ***surface***

Intersection method:

```
if (!bvol.intersect(ray,t))  
    return false;  
else  
    return surface.intersect(ray,t);
```

This change is transparent to the renderer (only it might run faster).

Implementing a bounding volume hierarchy

A ***BoundedSurface*** can contain a ***surfaces*** list.

Any ***surface*** in this list might also be a ***BoundedSurface***

This makes the ***BoundedSurface*** a recursive hierarchical structure

Or a Bounding Volume Hierarchy

Spatial Data Structures

Basic Idea – asymptotic improvement in spatial queries by subdividing

Two types of subdivisions – ***object partitioning scheme*** and ***space partitioning scheme***

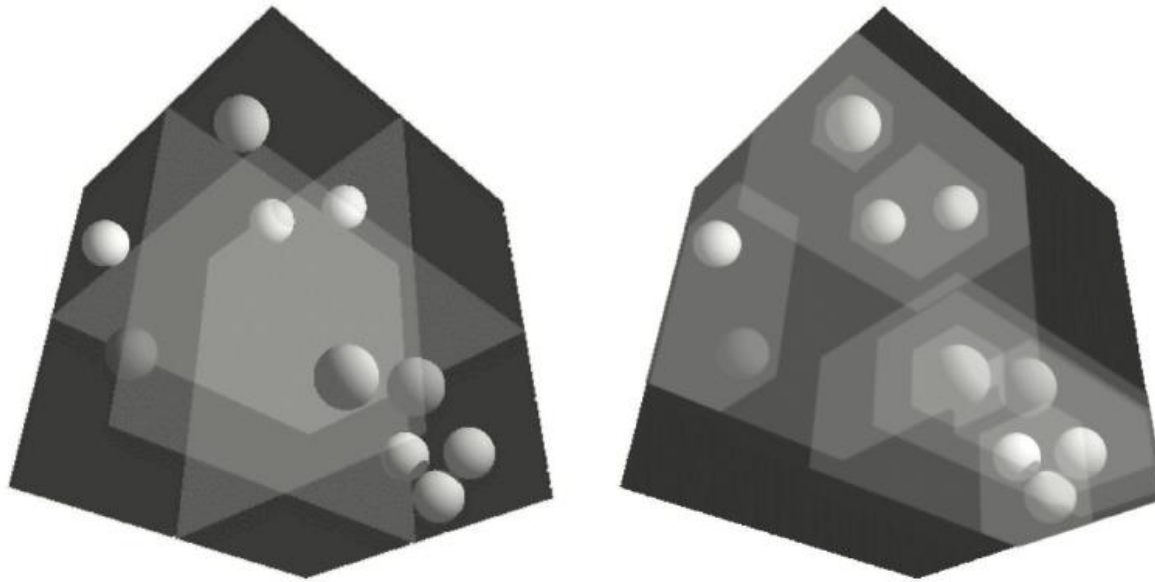


Figure 12.22. Left: a uniform partitioning of space. Right: adaptive bounding-box hierarchy.
Image courtesy David DeMarle.

AABB Tree Construction

Make AABB for whole scene/object, then split into two parts

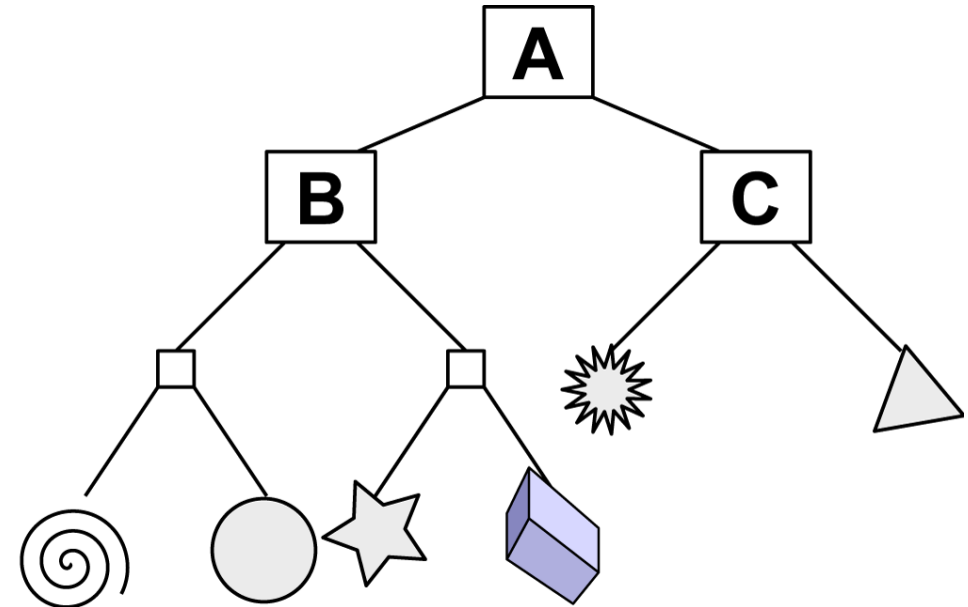
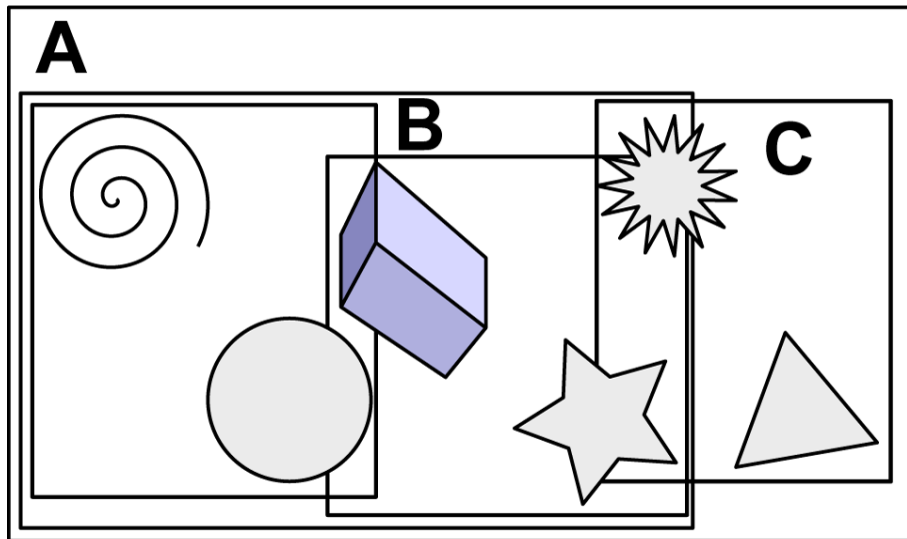
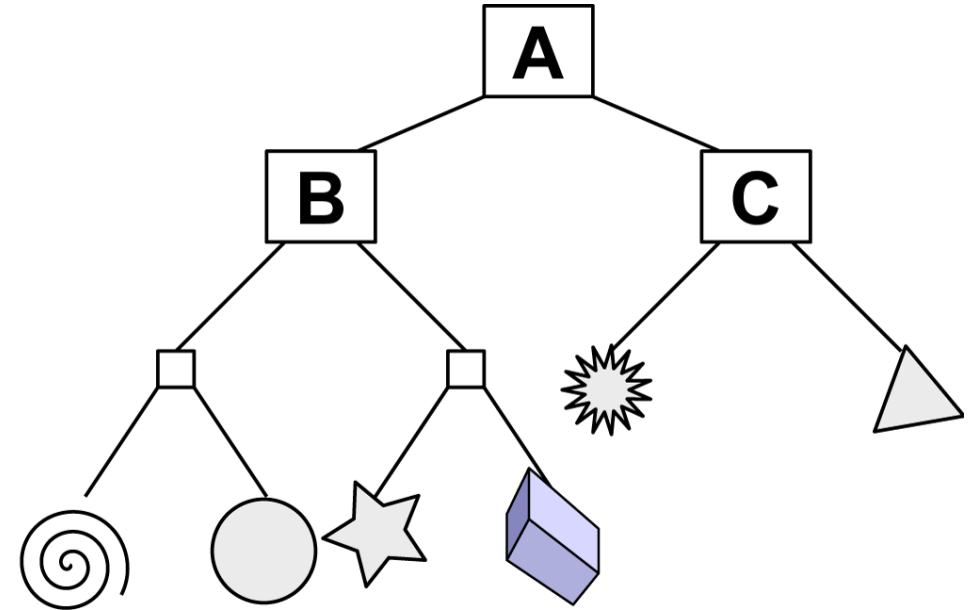
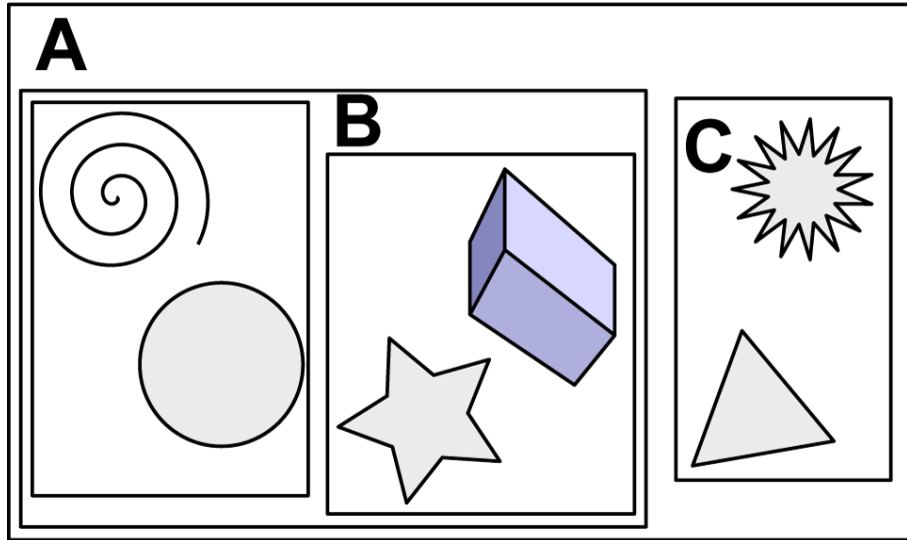
- Recurse on parts.
- Stop when there is one (or a few) object/triangle in your box.

How to split into parts?

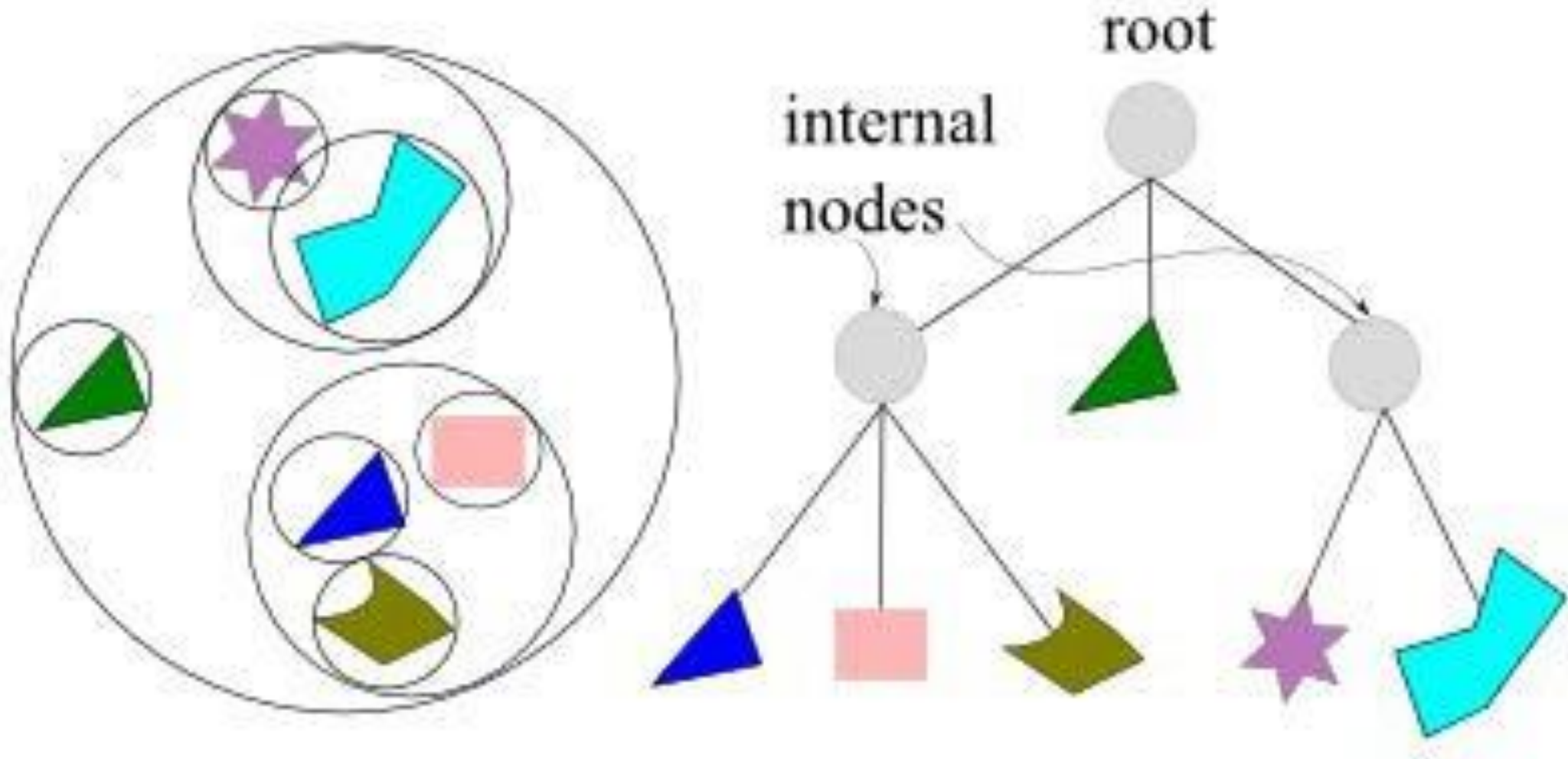
Space based: partition objects based on value relative to the center of longest dimension.

Object based: sort the objects along the longest dimension and divide them equally.

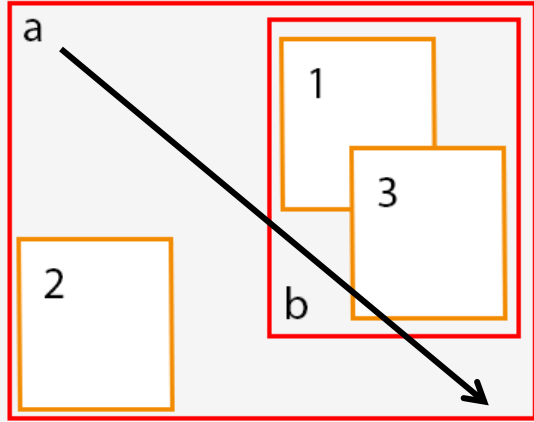
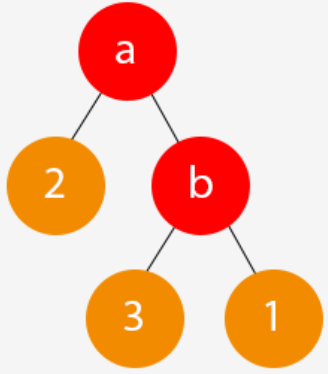
AABB Tree Construction (object based)



Sphere Trees



Ray and AABB tree Intersection



```
bvh::intersect(ray,t)
{
    if (aabb== null || !aabb.intersect(ray,t))
        return false;
    else
    {
        i1=left.intersect(ray,t1);
        i2=right.intersect(ray,t2);
        if (i1 && i2) {t=min(t1,t2); return true;}
        if (i1) {t=t1; return true;}
        if (i2) {t=t2; return true;}
        return false;
    }
}
```

DFS traversal of tree nodes!

BVH Distance Queries

```
minDistance(bvNode, point, currentMin) {  
    if (isLeaf(bvNode)) {  
        d1=d2=minDist(bvNode.object, point);  
    } else {  
        d1=minDistance(bvNode.left, point, currentMin);  
        d2=minDistance(bvNode.right, point, currentMin);  
    }  
    if (min(d1,d2) > currentMin) {  
        return currentMin;  
    }  
    return min(d1,d2)  
}
```

Is DFS traversal of tree nodes efficient?

BFS with a priority queue!

BVH Intersection Queries

```
leaf_pairs  $\leftarrow$  {};  
if (root_A.box  $\cap$  root_B.box) Q.insert(root_A, root_B );  
while Q not empty {  
    {nodeA,nodeB}  $\leftarrow$  Q.pop;  
    if (nodeA and nodeB are leaves) leaf_pairs.insert( node_A, node_B );  
    else if (node_A is a leaf) { /* symmetrically for node_B */  
        if (node_A.box  $\cap$  node_B.left.box) Q.insert( node_A, node_B.left ); /* symmetrically for node_B.right */  
    else {  
        if (node_A.left.box  $\cap$  node_B.box) Q.insert( node_A.left, node_B);  
        if (node_A.right.box  $\cap$  node_B.box) Q.insert( node_A.right, node_B);  
        if (node_A.box  $\cap$  node_B.left.box) Q.insert( node_A, node_B .left);  
        if (node_A.box  $\cap$  node_B.right.box) Q.insert( node_A.left, node_B.right);  
    }  
}
```

Triangle-Triangle intersection

T_1 intersects $T_2 \iff$ at least one tri edge intersects the other tri.

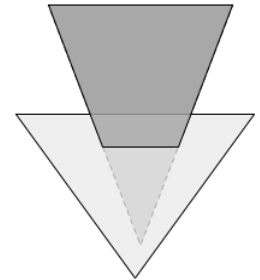
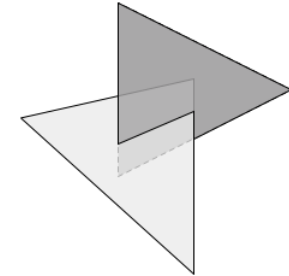
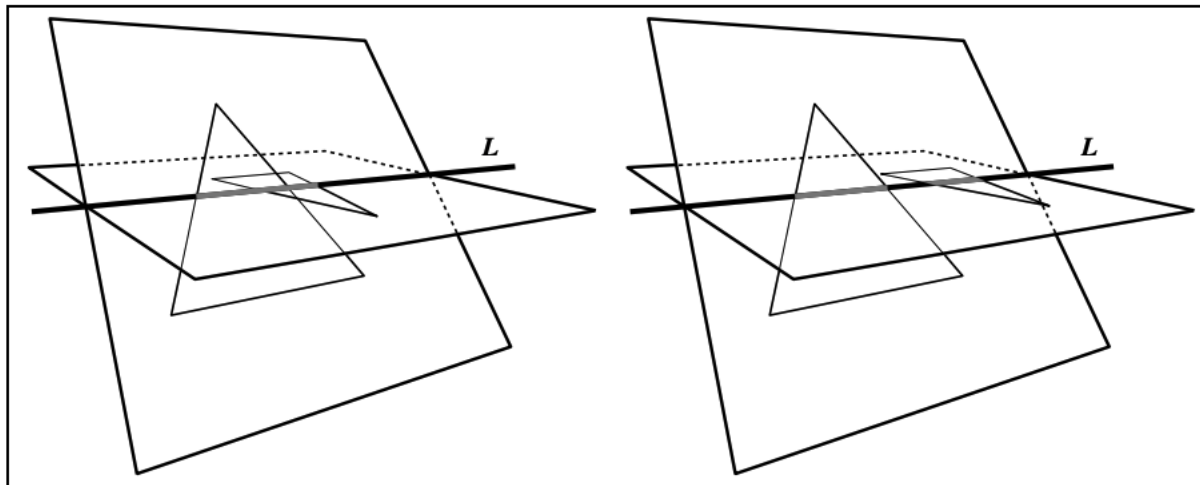
Algorithm 1: Test edge-tri intersection for all 6 edges.

T_1 intersects $T_2 \Rightarrow$ Vertices of T_1, T_2 straddle plane of T_2, T_1 respectively.

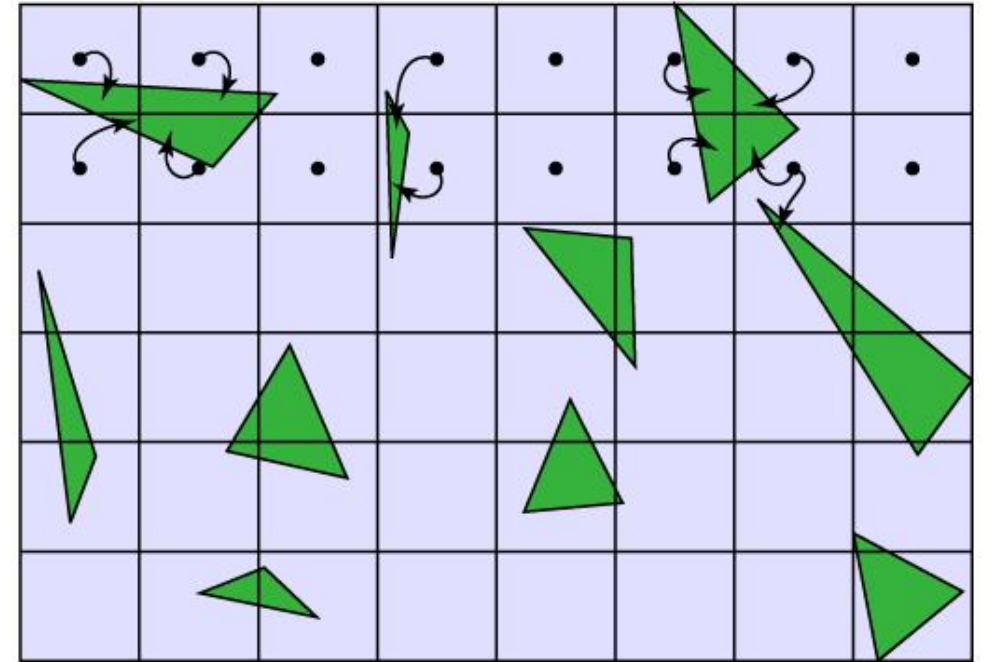
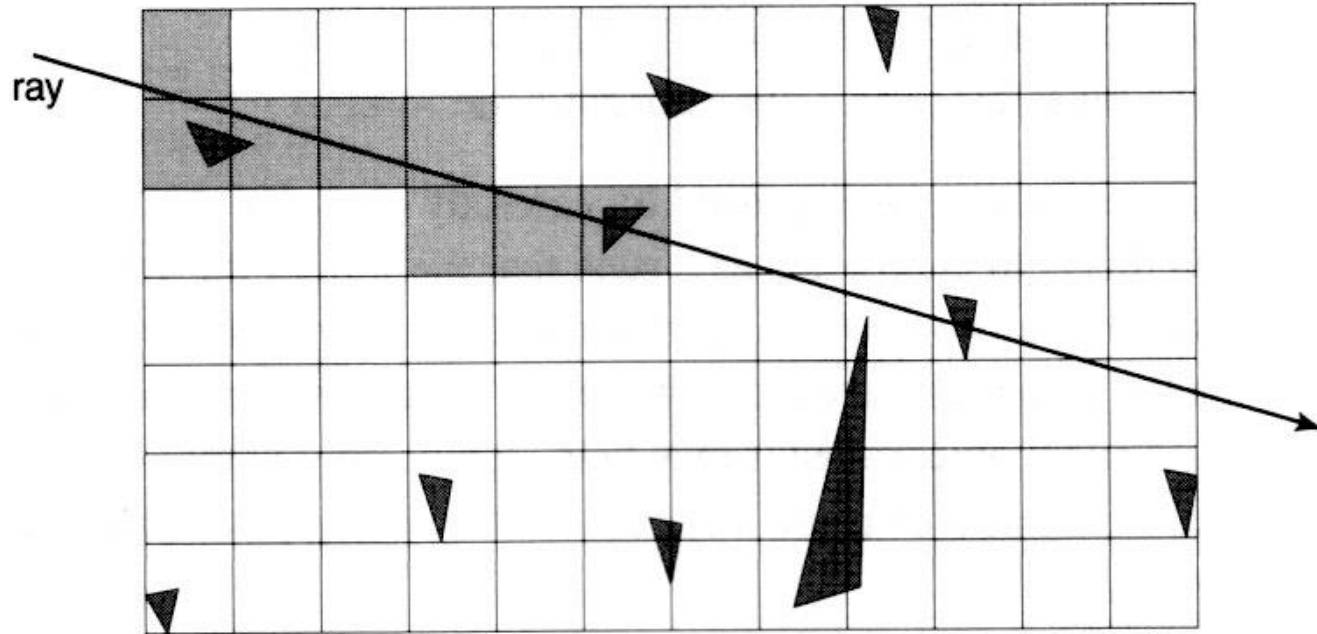
What if T_1, T_2 are co-planar?

Algorithm 2: if (vertices of T_1, T_2 on the same side of plane of T_2, T_1) return false;

The triangles intersect iff the triangle intervals along line of intersection do.



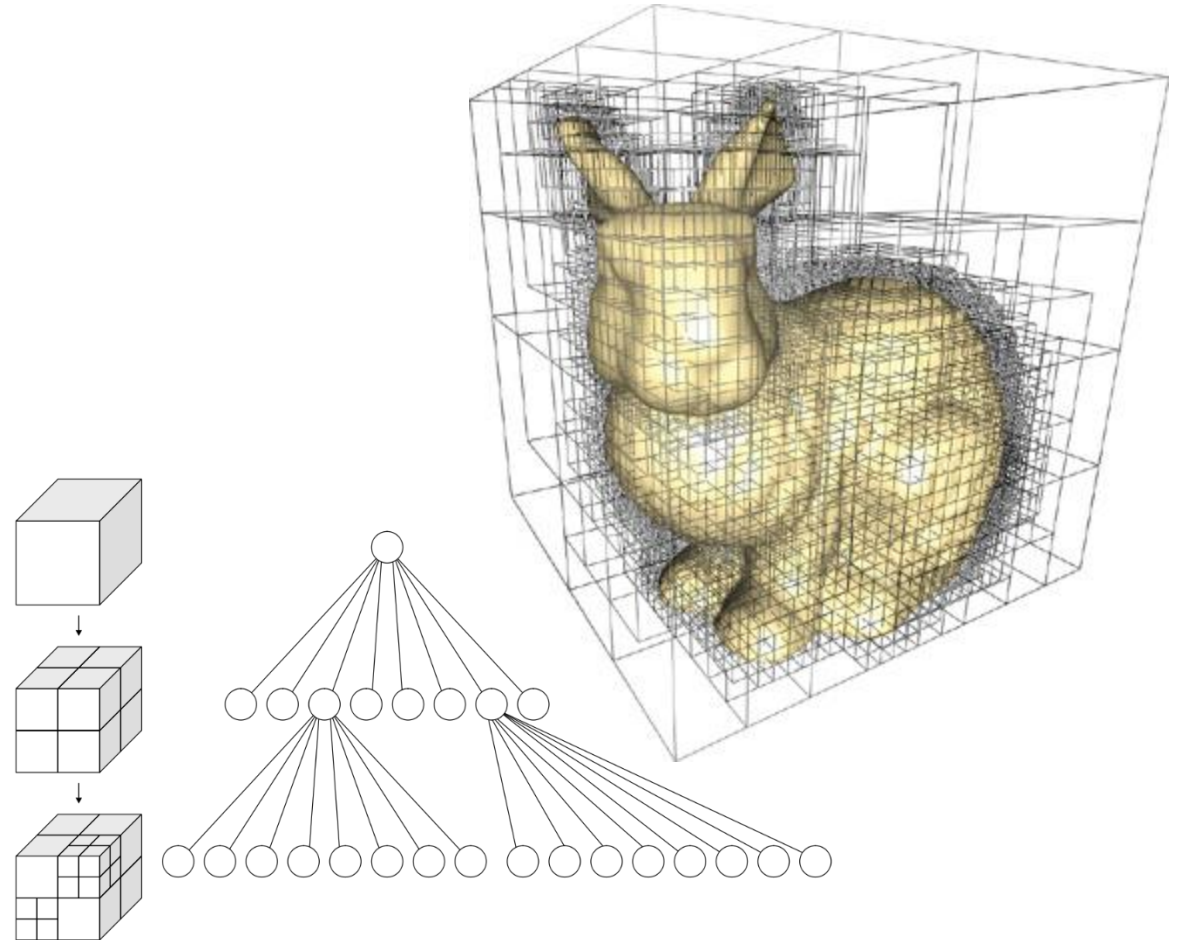
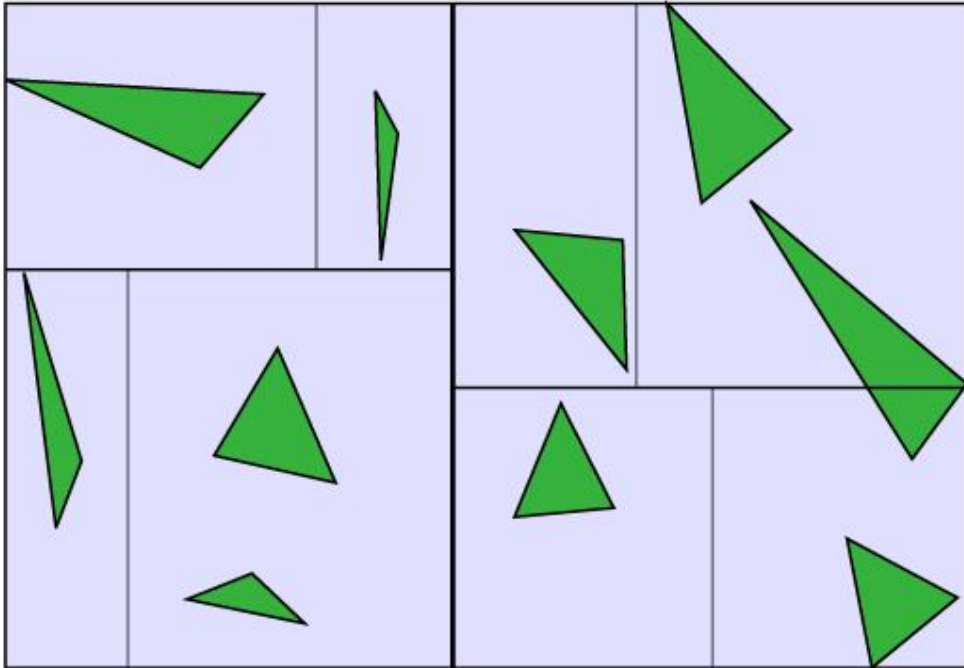
Regular space subdivision



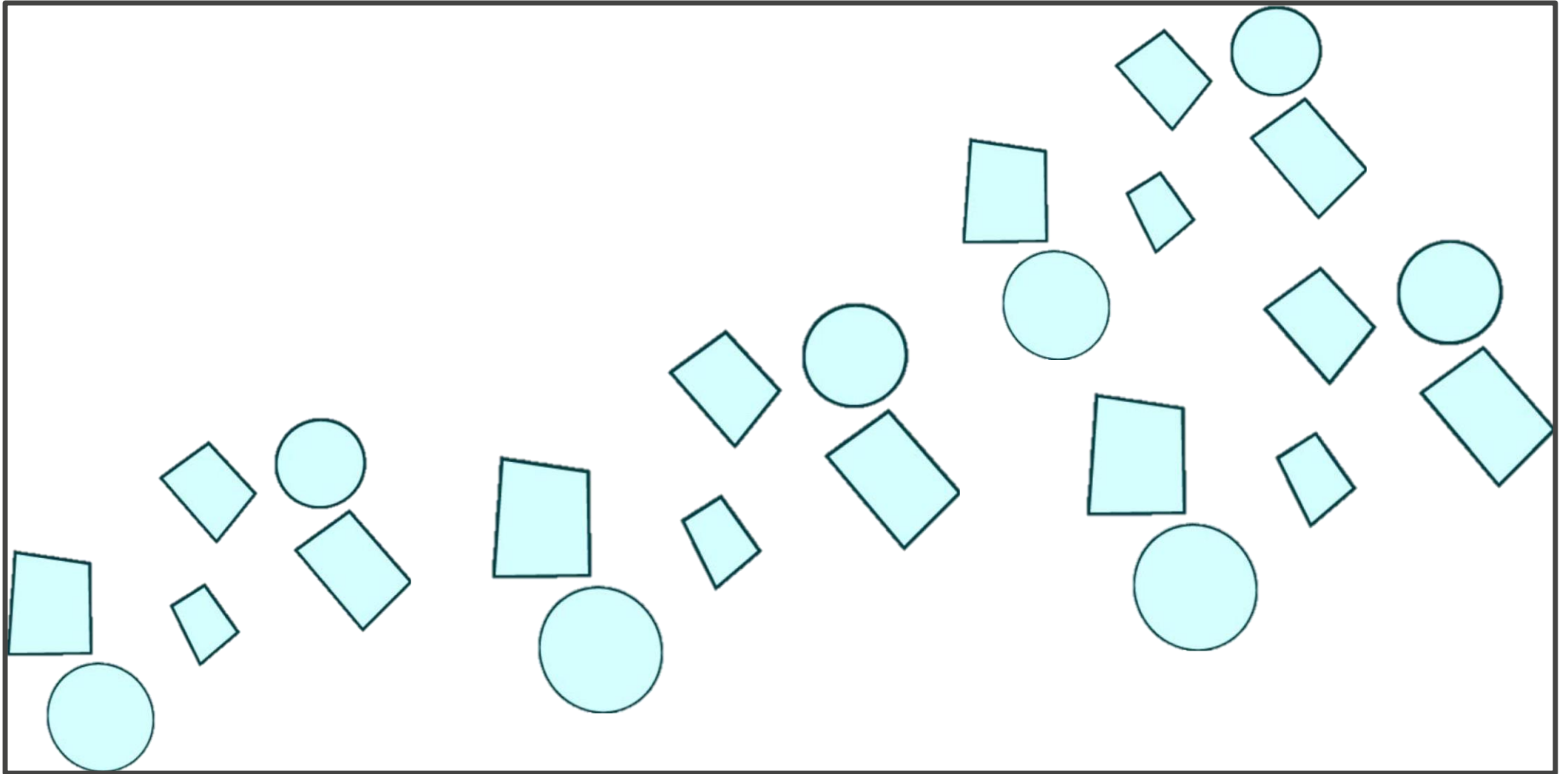
Grid divides space, not objects.

Non-regular space subdivision

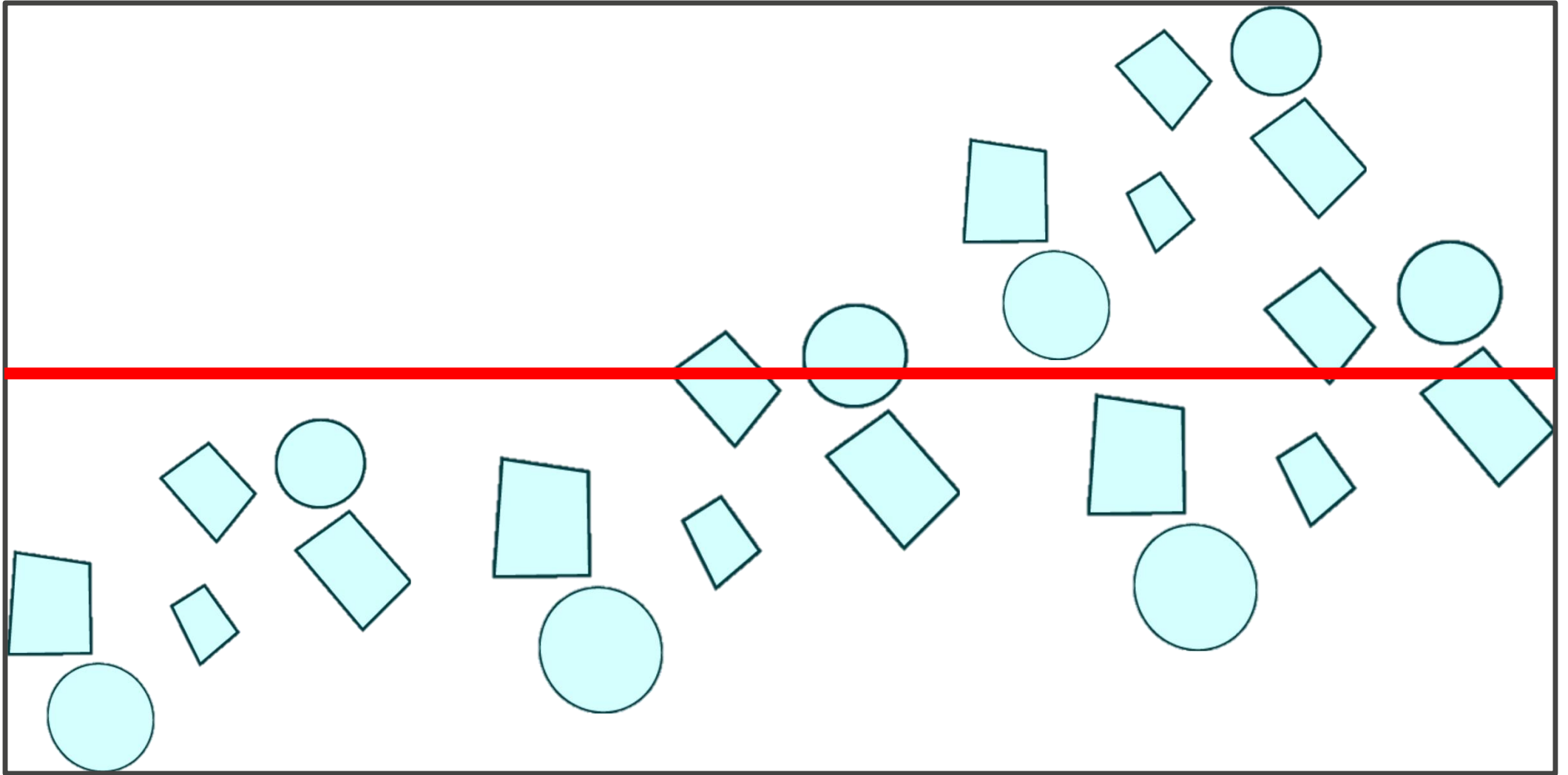
k -d Tree
Octree



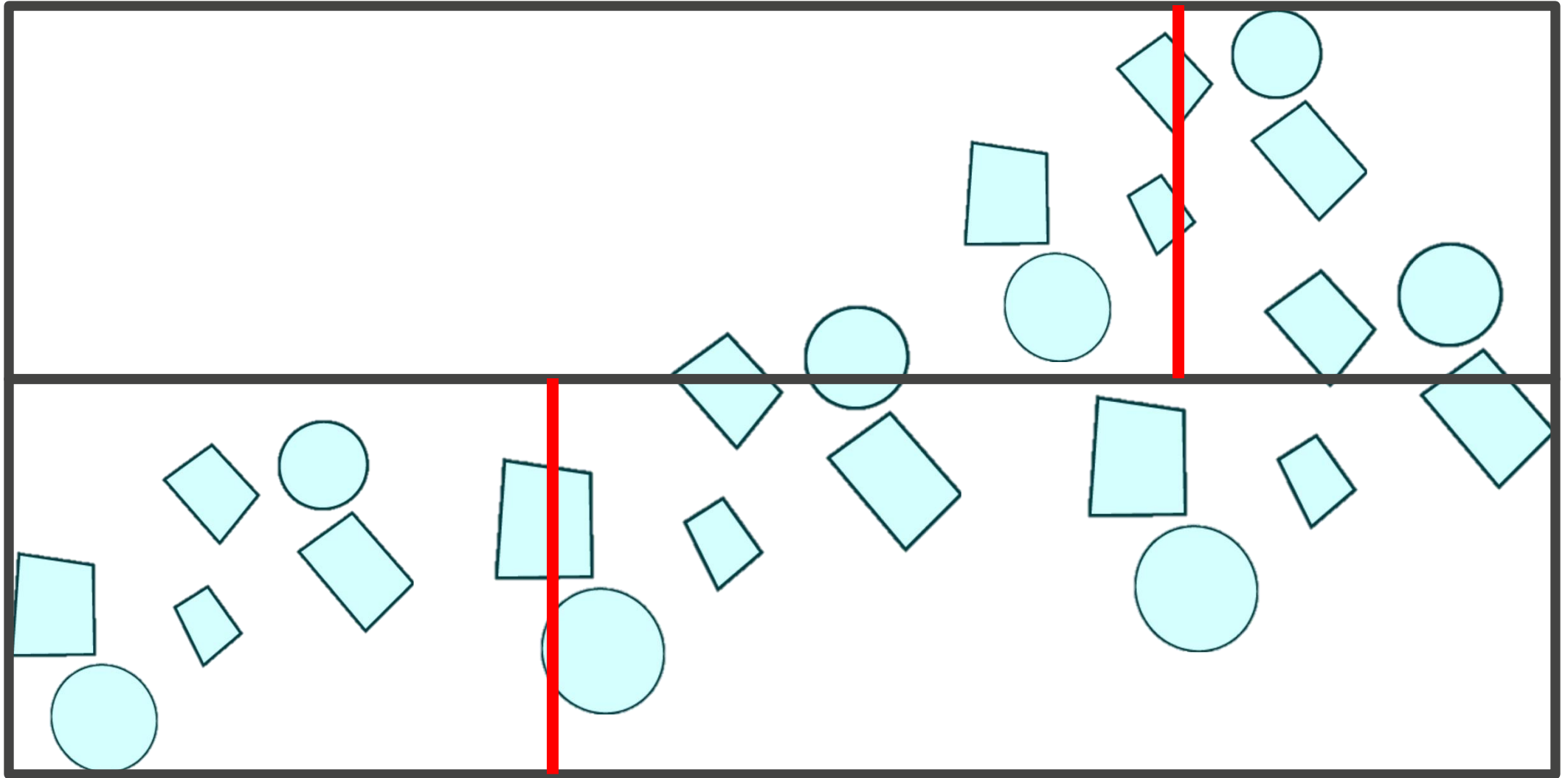
Constructing a k-d Tree



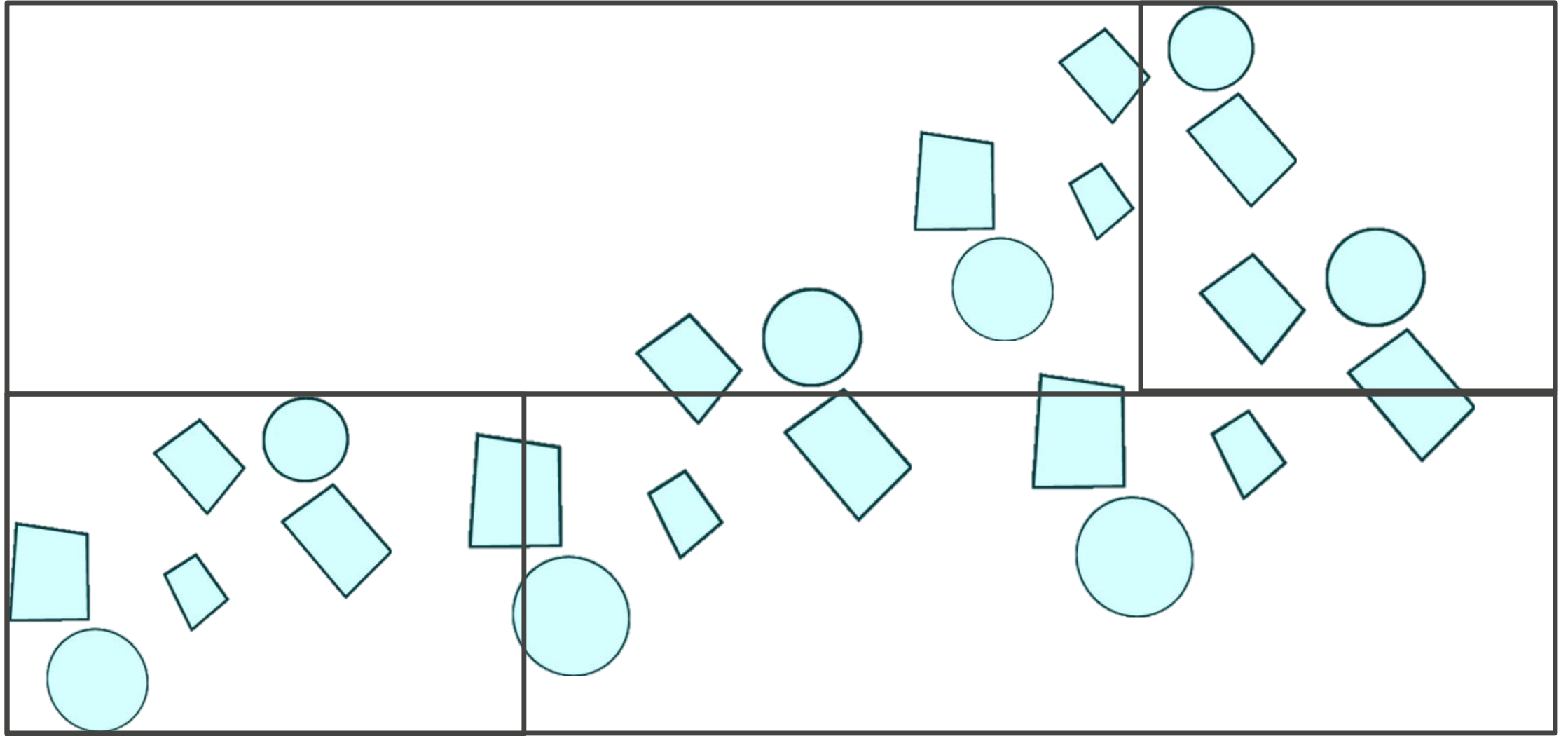
Constructing a k-d Tree



Constructing a k-d Tree



Constructing a k-d Tree



Comparison of Data Structures

The cost of maintaining the data structures?

The benefits from reduced intersection testing?

Dynamic, animated scenes. Objects move around. Meshes deform.

- BVHs: cost for re-bounding the objects, tree structure is easy to change, objects, object-object intersection testing

Static Scenes

- Space Partitioning: not much rebuilding, skipping empty spaces, can utilize higher ray-count.

Check your geometry (grid-like vs organic shapes)

Assignment #4 Tasks

- **src/ray_intersect_triangle.cpp** Intersect a ray with a triangle (feel free to crib your solution from the ray casting).
- **src/ray_intersect_triangle_mesh_brute_force.cpp** Shoot a ray at a triangle mesh with faces and record the closest hit. Use a brute force loop over all triangles. This will be your reference solution.
- **src/ray_intersect_box.cpp** Intersect a ray with a solid box
- **src/insert_box_into_box.cpp** Grow a box B by inserting a box A.
- **src/insert_triangle_into_box.cpp** Grow a box B by inserting a triangle with corners a, b, and c.
- **src/AABBTree.cpp** Construct an axis-aligned bounding box tree given a list of objects. Use the midpoint along the longest axis of the box containing the given objects to determine the left-right split.
- **src/AABBTree_ray_intersect.cpp** Determine whether and how a ray intersects the contents of an AABB tree. The method should perform in $\log$ time for a tree containing reasonably distributed objects.
- **src/nearest_neighbor_brute_force.cpp** Compute the nearest neighbor for a query in the set of points (rows of points). This should be a slow reference implementation.
- **src/point_box_squared_distance.cpp** Compute the squared distance between a query point and a box.
- **src/point_AABBTree_squared_distance.cpp** Compute the distance from a query point to the objects stored in a AABBTree using a priority queue.
- **src/triangle_triangle_intersection.cpp** Determine whether two triangles intersect.
- **src/box_box_intersect.cpp** Determine if two bounding boxes intersect.
- **src/find_all_intersecting_pairs_using_AABBTrees.cpp** Find all intersecting pairs of leaf boxes between one AABB tree and another.